

第九章

门电路与组合逻辑电路

9.1 数字电路概述

9.2 逻辑代数与逻辑函数

9.3 集成逻辑门电路

9.4 组合逻辑电路的分析与设计

9.5 常用组合逻辑模块

9.6 应用举例

1

学习目标

- 了解数字信号和数字电路的特点。
- 掌握与、或、非三种基本逻辑运算以及与非、异或等常用逻辑运算的逻辑功能。
- 了解逻辑代数的基本运算法则和基本定律。
- 掌握应用逻辑代数运算法则和卡诺图进行逻辑化简的方法。
- 掌握几种逻辑函数表示形式之间的转换方法。
- 了解集成逻辑门电路的特点和使用中的实际问题。
- 掌握组合逻辑电路的分析和设计的方法。
- 掌握常用中规模组合逻辑模块的使用方法。

9.1 数字电路概述

9.1.1 脉冲信号和数字信号

9.1.2 数字电路中常用数制



3

9.1.1 脉冲信号和数字信号

数字电路：处理数字信号的电路。

电子电路中的信号

模拟信号 随时间连续变化的信号。

如：正弦波、锯齿波信号等。

数字信号 时间和幅度都是离散的。

如：脉冲信号等。

1. 脉冲信号

脉冲幅度: A

脉冲前沿: t_r

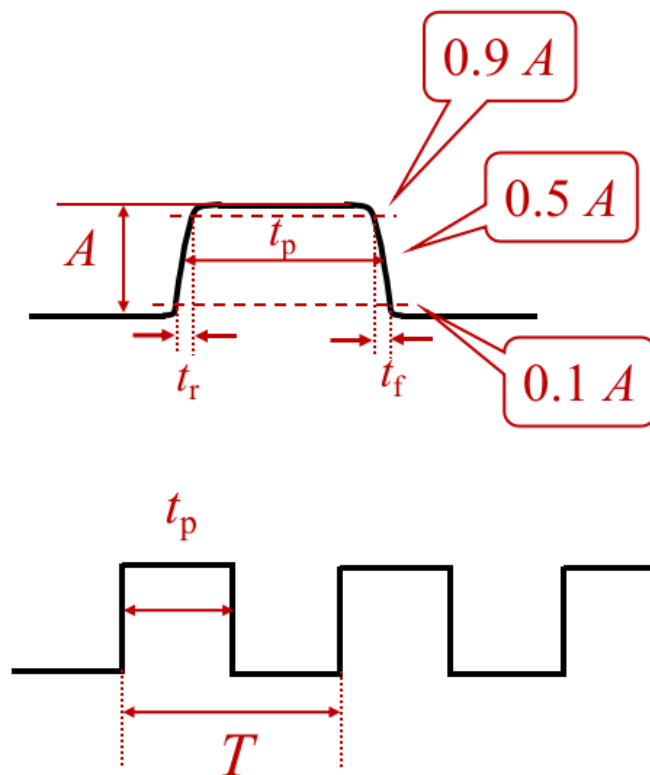
脉冲后沿: t_f

脉冲宽度: t_p

脉冲周期: T

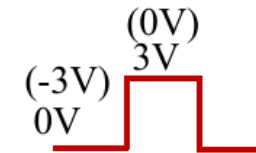
脉冲频率: $f=1/T$

占空比: $q(\%) = t_p/T \times 100\%$

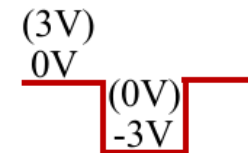


正、负脉冲信号

正脉冲： 跃变后的电位比跃变前高



负脉冲： 跃变后的电位比跃变前低

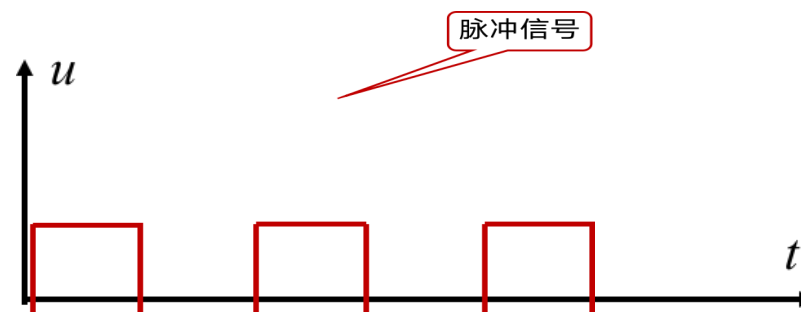


2. 数字信号

脉冲信号

产品数量的统计。
数字表盘的读数。

数字电路信号：

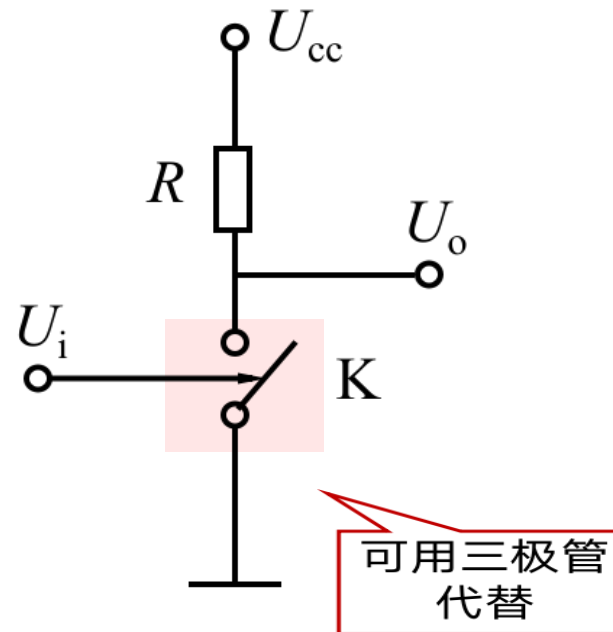




研究数字电路时注重**电路输出、输入间的逻辑关系**，因此不能采用模拟电路的分析方法。主要的工具是逻辑代数，**电路的功能用逻辑状态表（真值表）、逻辑表达式及波形图表示。**



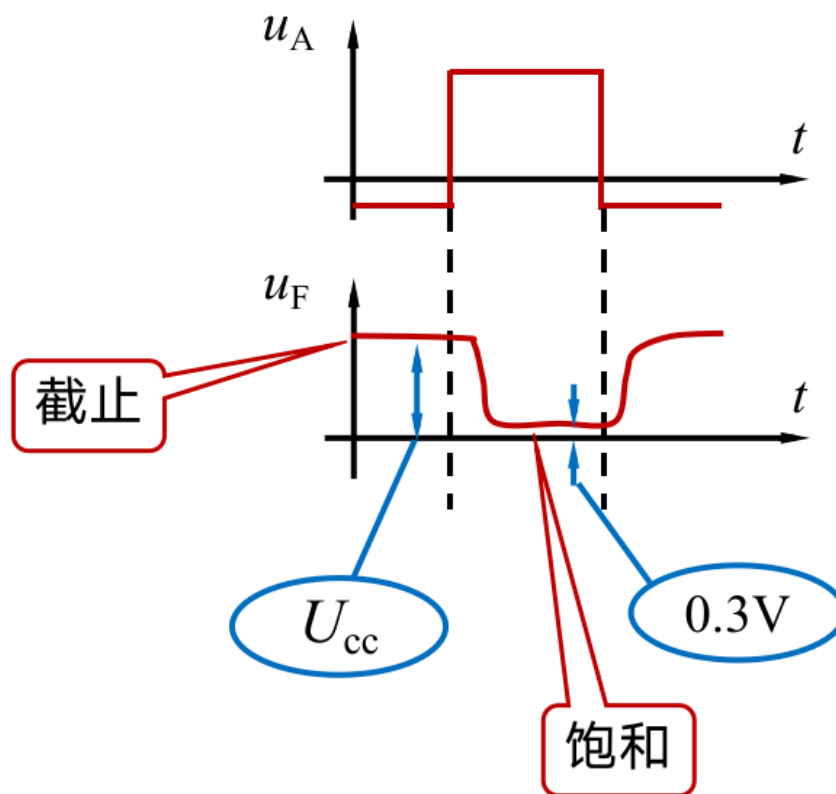
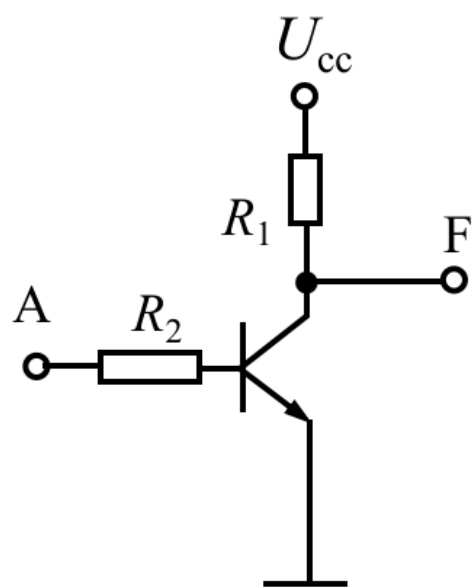
在数字电路中，三极管工作在开关状态，即工作在**饱和和截止状态**。



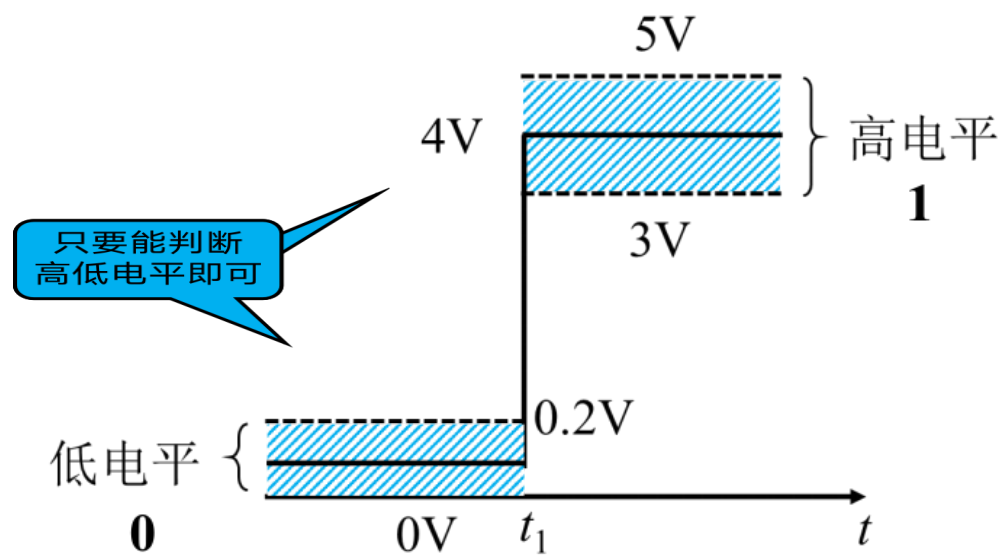
K开----- $U_o=1$, 输出高电平

K合----- $U_o=0$, 输出低电平

三极管的开关特性:



在数字电路中，一般用高电平代表1、低电平代表0，即所谓的
正逻辑系统。



11

9.1.2 数字电路中常用数制

1. 进位计数制

十进制：以十为基数的计数体制

表示数的十个数码：

0、1、2、3、4、5、6、7、8、9

位权： 10^n

遵循**逢十进一**的规律 $(N)_{10} = \sum_{i=-m}^{n-1} a_i \times 10^i$

$$1976.5 = 1 \times 10^3 + 9 \times 10^2 + 7 \times 10^1 + 6 \times 10^0 + 5 \times 10^{-1}$$

二进制：以二为基数的计数体制

表示数的两个数码：

0、1

位权： **2^n**

遵循**逢二进一**的规律

$$(N)_2 = \sum_{i=-m}^n a_i \times 2^i$$

$$(1001)_2 = 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = (9)_{10}$$

优缺点

- ✓ 用电路的两个状态---开、关来表示二进制数，数码的存储和传输简单、可靠。
- ✓ 位数较多，使用不便；不合人们的习惯，输入时将二进制转换成二进制，运算结果输出时再转换成十进制数。

2. 二进制与十进制之间的转换

二进制转换为十进制

按权展开

$$(1011)_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = (11)_{10}$$

十进制转换为二进制

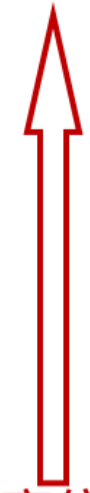
求商取余

$$(25)_{10} = (11001)_2$$

十进制与二进制之间的转换，可以用二除十进制数，余数是二进制数的第0位(K_0)，然后依次用二除所得的商，余数依次是第一位(K_1)、第二位(K_2)、……。

15

转换过程：

2	25	余 1		K_0	低位
2	12	余 0		K_1	
2	6	余 0		K_2	
2	3	余 1		K_3	
2	1	余 1		K_4	
	0				高位

$$(25)_{10} = (11001)_2$$

3. 二—十进制 (BCD码)

二进制编码的十进制数
(Binary Coded Decimal Codes)

最常用的BCD码：8421码

BCD码				十进制数
0	0	0	0	0
0	0	0	1	1
0	0	1	0	2
0	0	1	1	3
0	1	0	0	4
0	1	0	1	5
0	1	1	0	6
0	1	1	1	7
1	0	0	0	8
1	0	0	1	9
“8”	“4”	“2”	“1”	



17

9.2 逻辑代数与逻辑函数

9.2.1 逻辑代数

9.2.2 逻辑函数及其表示法

9.2.3 逻辑函数的化简与变换



18

9.2.1 逻辑代数

在数字电路中，我们要研究的是电路的输入输出之间的逻辑关系，所以数字电路又称**逻辑电路**，相应的研究工具是**逻辑代数（布尔代数）**。

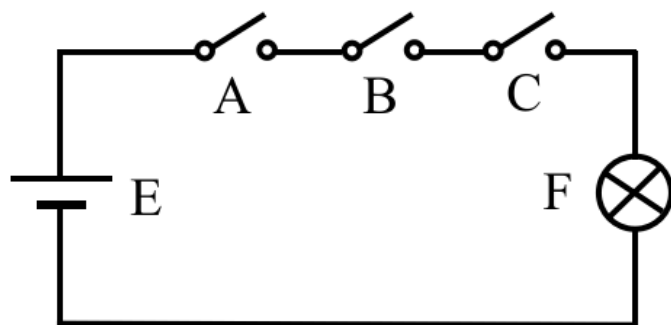
在逻辑代数中，逻辑函数的变量只能取两个值（**二值变量**），即0和1，中间值没有意义，这里的0和1只表示两个对立的逻辑状态，如电位的低高（0表示低电位，1表示高电位）、开关的开合等。

1. 基本逻辑运算

(1) “与”逻辑运算和与门

与逻辑

A、B、C都具备时，事件F才发生。

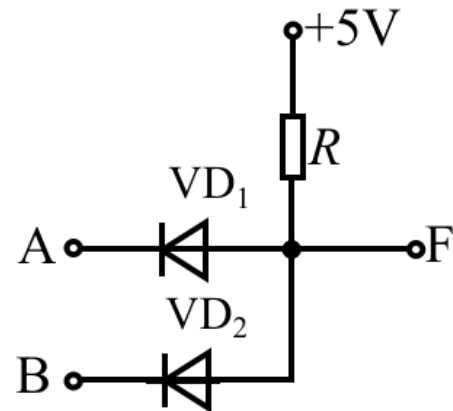


设 开关闭为“1”
开关开为“0”

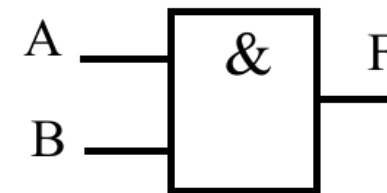
灯亮为“1”
不亮为“0”

则开关A、B、C与灯F的关系为“与”逻辑

二极管与门电路



逻辑符号



输入：A、B：3V $\leftrightarrow$ 逻辑“1”；0V $\leftrightarrow$ 逻辑“0”

输出：F：3V $\leftrightarrow$ 逻辑“1”；0V $\leftrightarrow$ 逻辑“0”

21

逻辑式

$$F=A \cdot B \cdot C$$

逻辑与
(逻辑乘)

逻辑状态表

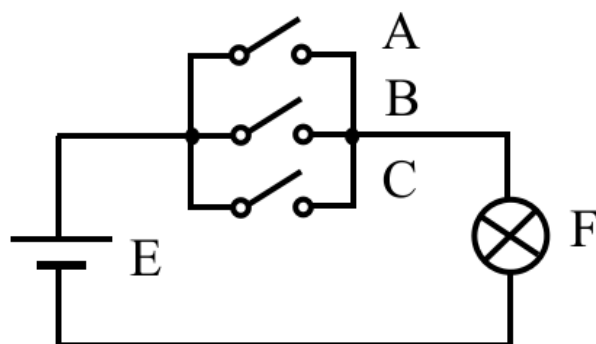
A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

有0有0出0

全1出1

(2) “或”逻辑运算和或门

或逻辑 A、B、C只有一个具备时，事件F就发生。

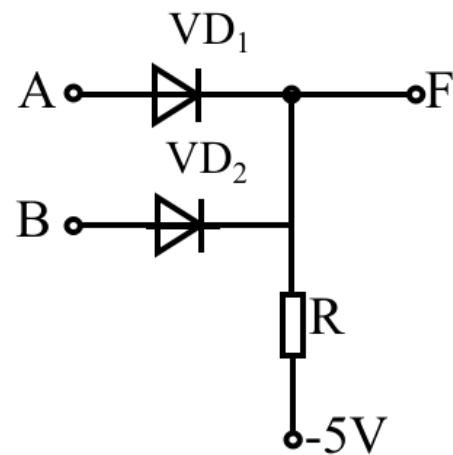


设 开关闭为“1”
开关开为“0”

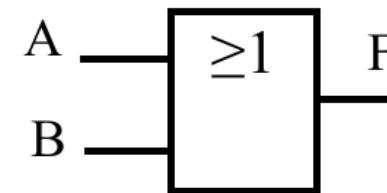
灯亮为“1”
不亮为“0”

则开关A、B、C与灯F的关系为“或”逻辑

二极管或门电路



逻辑符号



输入：A、B：3V ↔ 逻辑“1”；0V ↔ 逻辑“0”

输出：F：3V ↔ 逻辑“1”；0V ↔ 逻辑“0”

逻辑式

$$F=A+B+C$$

逻辑或
(逻辑加)

逻辑状态表

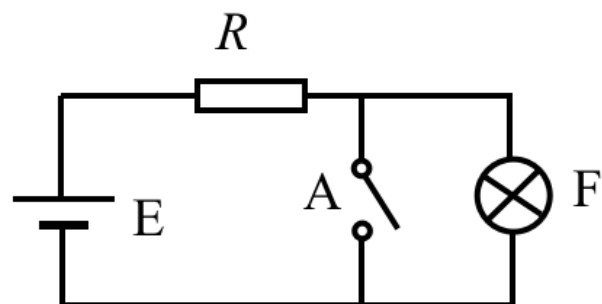
A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

全0全出0

有1出1

(3) “非”逻辑运算和非门

非逻辑 A具备时，事件F不发生；A不具备时，事件F发生。

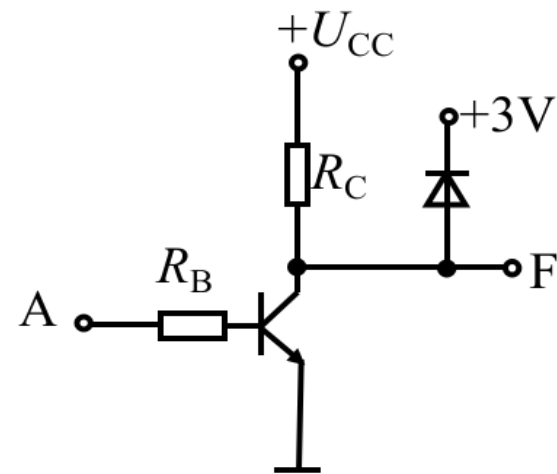


设 开关闭为 “1”
开关开为 “0”

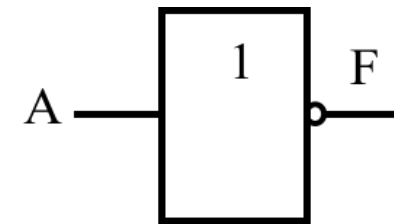
灯亮为 “1”
不亮为 “0”

则开关A与灯F的关系为 “非” 逻辑

三极管非门电路



逻辑符号



限幅二极管

输入：A： 3V ↔ 逻辑“1”； 0V ↔ 逻辑“0”

输出：F： 3V ↔ 逻辑“1”； 0V ↔ 逻辑“0”

27

逻辑式

$$F = \overline{A}$$

逻辑非
(逻辑反)

逻辑状态表

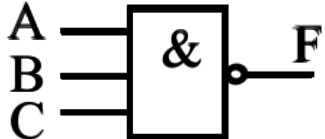
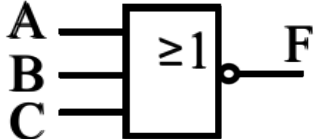
A	F
0	1
1	0

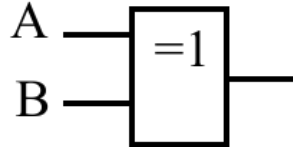
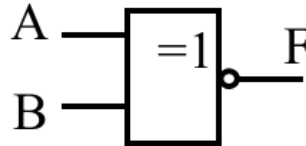
有0出1

有1出0

2. 复合逻辑运算

“与”、“或”、“非”是三种基本的逻辑关系，任何其它的逻辑关系都可以以它们为基础表示。

与非 ：条件A、B、C都具备，则F 不发生。	$F \equiv A \cdot \overline{B} \cdot \overline{C}$	
或非 ：条件A、B、C任一具备，则F 发生。	$F = \overline{A+B+C}$	

异或：输入不同时，输出为“1”，输入相同时，输出为“0”。	$F = \overline{A}B + A\overline{B}$	
同或：输入不同时，输出为“0”，输入相同时，输出为“1”。	$F = AB + \overline{A}\overline{B}$	

3. 逻辑代数的基本定律

0-1律

$$A \cdot 0 = 0$$

$$A + 1 = 1$$

自等律

$$A \cdot 1 = A$$

$$A + 0 = A$$

重叠律

$$A \cdot A = A$$

$$A + A = A$$

互补律

$$A \cdot \bar{A} = 0$$

$$A + \bar{A} = 1$$

还原律

$$\overline{\bar{A}} = A$$

所以，可以得到以下逻辑运算：

$$0 \cdot 0 = 0 \cdot 1 = 1 \cdot 0 = 0$$

$$0 + 0 = 0$$

$$1 \cdot 1 = 1$$

$$\bar{1} = 0$$

$$0 + 1 = 1 + 0 = 1 + 1 = 1$$

$$\bar{0} = 1$$

31

交换律

$$A+B=B+A$$

$$A \cdot B=B \cdot A$$

结合律

$$A+(B+C)=(A+B)+C=(A+C)+B$$

$$A \cdot (B \cdot C)=(A \cdot B) \cdot C$$

分配律

$$A(B+C)=A \cdot B+A \cdot C$$

$$A+B \cdot C=(A+B)(A+C)$$

普通代数
不适用!

反演律 $A \cdot \overline{B} = \overline{A} + \overline{B}$

$$A \overline{+} B = \overline{A} \cdot \overline{B}$$

可以用列真值表的方法证明：

A	B	$A \cdot B$	$A \cdot \overline{B}$	$\overline{A}$	$\overline{B}$	$\overline{A} + \overline{B}$
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0



吸收律

$$A+AB=A$$

$$A \cdot B + A \cdot \bar{B} = A$$

$$A + A \cdot B = A$$

$$(A+B) \cdot (A+\bar{B}) = A$$

$$A \cdot (A+B) = A$$

$$A \cdot (\bar{A}+B) = A \cdot B$$

$$A + \bar{A} \cdot B = A + B$$

$$AB + \bar{A}C + BC = AB + \bar{A}C$$

证明: $A+AB=A(1+B)=A \cdot 1=A$

例如: $AB + CD + \overline{AB\bar{D}}(E+F) = AB + CD$

被吸收



34

9.2.2 逻辑函数及其表示法

1. 逻辑函数

任何一个具体的逻辑因果关系都可以用一个确定的逻辑函数来描述。

2. 逻辑函数的表示法

(1) 逻辑函数式

把逻辑函数的输入、输出关系写成与、或、非等逻辑运算的组合式，即逻辑代数式，称为逻辑函数式，我们通常采用“与或”的形式。

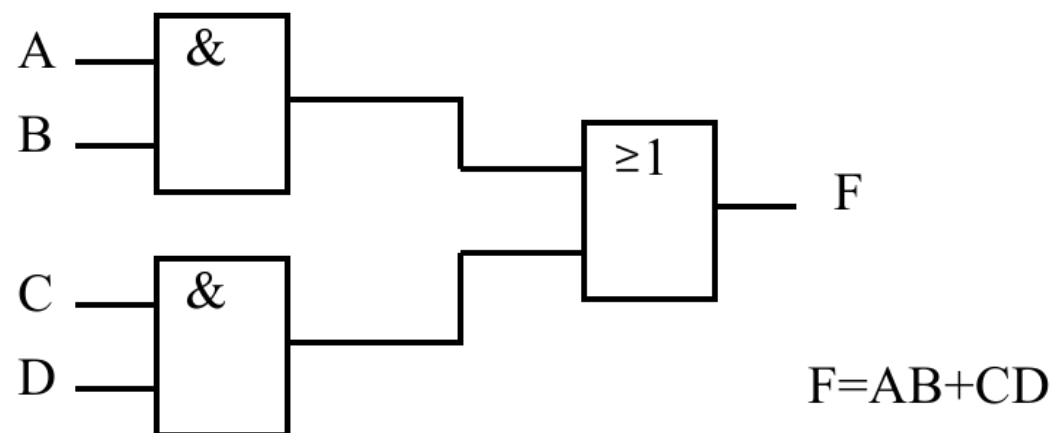
比如： $F = A\bar{B}\bar{C} + \bar{A}B\bar{C} + \bar{A}\bar{B}C + A\bar{B}C + ABC$

(2) 真值表 将输入、输出的所有可能状态一一对应地列出。

A	B	C	F
0	1	0	0
0	1	1	0
0	0	0	0
0	0	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

n 个变量可以有 2^n 个组合，一般按二进制的顺序，输出与输入状态一一对应，列出所有可能的状态。

(3) 逻辑图 把相应的逻辑关系用逻辑符号和连线表示出来。



3. 逻辑函数表示形式的转换

(1) 由真值表转换到与或表达式

第一步：取真值表中函数值为“1”的各项，将变量写成“与”的形式；（变量为1，取其本身，变量为0，取其反）

第二步：将各项写成“或”的形式。

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

$$F = \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC$$

(2) 由逻辑表达式转换到真值表

第一步：把逻辑表达式中变量的各种取值组合有序地添入真值表中；
(有 n 个变量时，变量的取值组合有 2^n 个)

第二步：计算出变量的各种取值组合对应的函数值，并添入表中。

A	B	F
0	0	0
0	1	1
1	0	1
1	1	0

$F = \overline{A}B + A\overline{B}$

(3) 逻辑表达式与逻辑图的转换

前面已经提到，在此不再重复！



39

9.2.3 逻辑函数的化简与变换

1. 逻辑函数的化简

在实现同一逻辑功能的前提下，逻辑式越简单，则需要门的数量越少，电路越简单。所以逻辑式的化简是分析和设计逻辑电路必不可少的步骤。

(1) 代数化简法

反复运用逻辑代数公式，以消去多余的乘积项和每个乘积项中多余的因子，使函数式得到化简。

例1

$$F = AB + \overline{A}\overline{B} + BC + \overline{B}\overline{C}$$

$$= (AB + \overline{A}\overline{B}) + (BC + \overline{B}\overline{C})$$

反演

$$= AB + \overline{A}\overline{B}(\underline{C + \overline{C}}) + (\underline{A + \overline{A}})BC + \overline{B}\overline{C}$$

配项

被吸收

$$= \underline{AB} + \overline{A}\overline{B}C + \overline{A}\overline{B}\overline{C} + \underline{ABC} + \overline{A}BC + \overline{B}\overline{C}$$

被吸收

$$= AB + \overline{A}C(\overline{B} + B) + \overline{B}\overline{C}$$

$$= AB + \overline{A}C + \overline{B}\overline{C}$$

例2

$$F = A\bar{B}C + \underline{AB\bar{C}} + ABC$$

提出AB

$$= A\bar{B}C + AB(\bar{C} + C)$$

=1

$$= \underline{A\bar{B}C} + AB$$

提出A

$$= A(\bar{B}C + B)$$

$$= A(\underline{C} + B)$$

反变量吸收

$$= AC + AB$$

思考

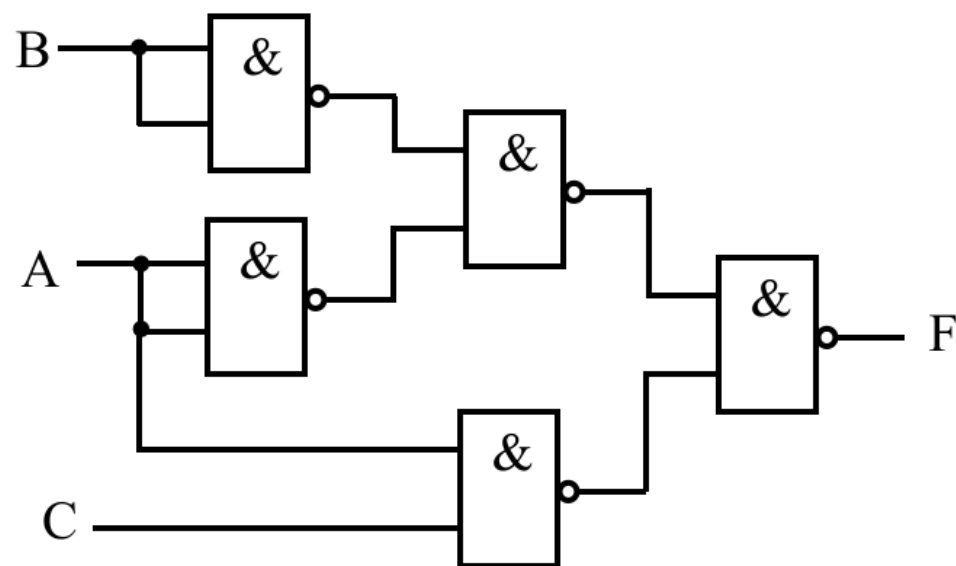
$$AB=AC \xrightarrow{?} B=C$$

$$A+B=A+C \xrightarrow{?} B=C$$

请注意与普通代数的区别！

例3 用与非门实现下列逻辑关系，画出逻辑图。

$$F = \bar{A} \bar{B} + AC = \overline{\overline{\bar{A} \bar{B} + AC}} = \overline{\bar{A} \bar{B}} \cdot \bar{AC}$$



(2) 卡诺图化简法

① 最小项与逻辑相邻

把逻辑函数的输入、输出关系写成与、或、非等逻辑运算的组合式，即逻辑代数式，称为逻辑函数式，我们通常采用“与或”的形式。

比如：
$$F = A\overline{B}\overline{C} + \overline{A}B\overline{C} + \overline{A}\overline{B}C + \overline{A}\overline{B}\overline{C} + ABC$$

若表达式中的乘积包含了所有变量的原变量或反变量，则这一项称为**最小项**，上式中每一项都是**最小项**。

若两个最小项只有一个变量以原、反区别，称它们**逻辑相邻**。

$$F = \boxed{A \overline{B} \overline{C}} + \overline{A} B \overline{C} + \overline{A} \overline{B} C + \boxed{\overline{A} B C} + ABC$$

逻辑相邻

$$\overline{A} B \overline{C} + A \overline{B} \overline{C} = \overline{B} \overline{C}$$

逻辑相邻的项可以
合并，消去一个因子

② 卡诺图 就是和变量的最小项对应的按一定规则排列的方格图，每一小方格填入一个最小项。

n 个输入变量有 2^n 个最小项，
卡诺图也就有 2^n 个小方格，
在卡诺图的行和列分别标出变量及其状态。

注意： 变量状态的次序是
00, 01, 11, 10

为了保证卡诺图中的各
最小项之间逻辑相邻。

画法： a.根据逻辑式或真值表画出卡诺图；
b.将逻辑式中的最小项或真值表中取“1”的最小项用
“1”填入卡诺图中相应的小方格内。

逻辑式原变量为“1”；反变量为“0”

47

例4 $Y = \underline{ABC} + \underline{AB\bar{C}} + \underline{\bar{A}BC} + \underline{A\bar{B}C}$ 用卡诺图表示。

BC A		BC			
		00	01	11	10
A	0			1	
	1		1	1	1

例5

$$F = \underline{\overline{A}} + \underline{\overline{A}\overline{B}} + \underline{BC\overline{D}} + \underline{B\overline{D}} \quad \text{用卡诺图表示。}$$

逻辑表达式不是最小项的形式，可直接填写，
也可化成最小项后，再画卡诺图。

CD \ AB		00	01	11	10
AB	00	1	1	1	1
	01	1	1	1	1
	11	1			1
	10				

例6 根据下面的真值表画出卡诺图。

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

		BC			
A	BC	00	01	11	10
0				1	
1			1	1	1

③ 应用卡诺图化简

方法：将卡诺图中所有取值为“1”的相邻小方格圈成矩形或方形，然后合并化简。

化简的原则：

- a. 相邻的小方格包括最上行与最下行、最左列与最右列、同行（列）两端的两个小方格。
- b. 圈的个数应尽可能的少，圈内的小方格应尽可能多。圈内小方格的个数应为 2^n 个。
- c. 圈内相邻的 2^n 项可以合并为一项，并消去 n 个因子。

保留相同变量，省略不同变量；“1”为原变量，“0”为反变量；然后各乘积项相加。

保证逻辑函数化到最简

51

例7

		CD			
		00	01	11	10
AB	00	1			1
	01		1	1	
	11	1	1	1	1
	10	1			1

$$F = \overline{B}\overline{D} + AB + BD$$

例8 根据下面的真值表画出卡诺图并化简。

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

BC		00	01	11	10
A	0			1	1
	1		1	1	

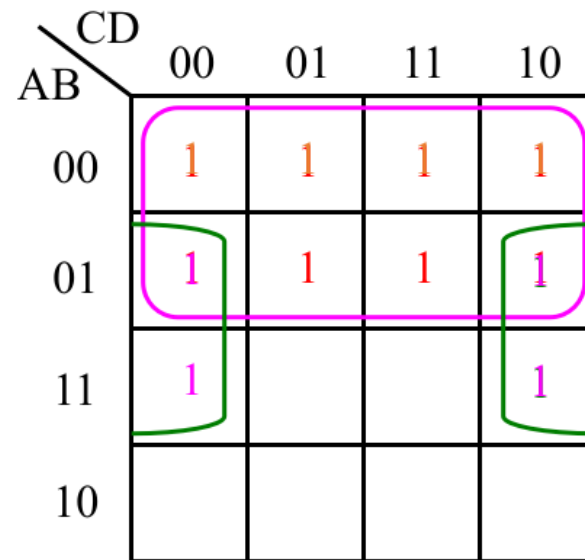
$$F = \overline{A}B + AC$$

例9 $F = \underline{A}BC + \underline{A}\underline{B}\bar{C} + \bar{\underline{A}}\underline{B}C + \underline{A}\bar{\underline{B}}\bar{C}$ 用卡诺图化简。

		BC			
		00	01	11	10
A	0			1	
	1	1		1	1

$$F = A\bar{C} + BC$$

例10 $F = \underline{\overline{A}} + \underline{\overline{A}\overline{B}} + \underline{BC\overline{D}} + \underline{B\overline{D}}$ 用卡诺图化简。



$$F = \overline{A} + B\overline{D}$$

2. 逻辑式的变换

在实现同一逻辑功能的前提下，逻辑式越简单，则需要门的数量越少，电路越简单。所以逻辑式的化简是分析和设计逻辑电路必不可少的步骤。

化简：

- (1) 根据逻辑代数的运算法则将逻辑式的项数减少，将每一项中的变量减少。
- (2) 根据要求将逻辑式转换为需要的逻辑运算形式。
如：“与非与非表达式”。



56

9.3 集成逻辑门电路

9.3.1 TTL与非门电路

9.3.2 CMOS门电路

9.3.3 三态输出与非门电路

9.3.4 集成逻辑门电路使用中的几个实际问题



57

逻辑门 组成数字逻辑电路的基本逻辑器件。

分立元件门电路 由二极管或晶体管组成的逻辑门电路。

- (1) 体积大、工作不可靠。
- (2) 需要不同电源。
- (3) 各种门的输入、输出电平不匹配。

集成门电路

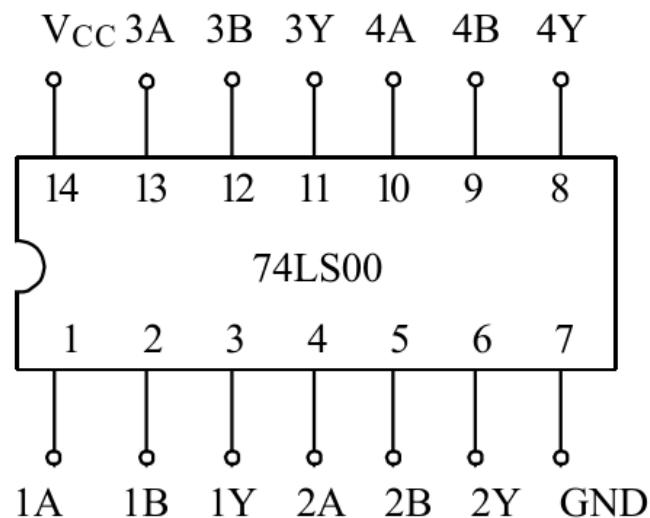
具有**体积小、可靠性高、速度快**的特点，而且输入、输出电平匹配，所以早已广泛采用。

按集成度：SSI，MSI，LSI，VLSI。

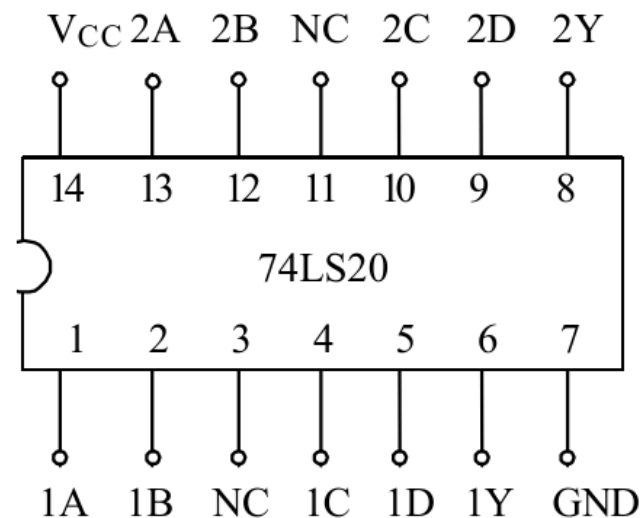
按制造工艺：TTL，MOS。

9.3.1 TTL与非门电路

74LS00内含4个2输入与非门，
74LS20内含2个4输入与非门。



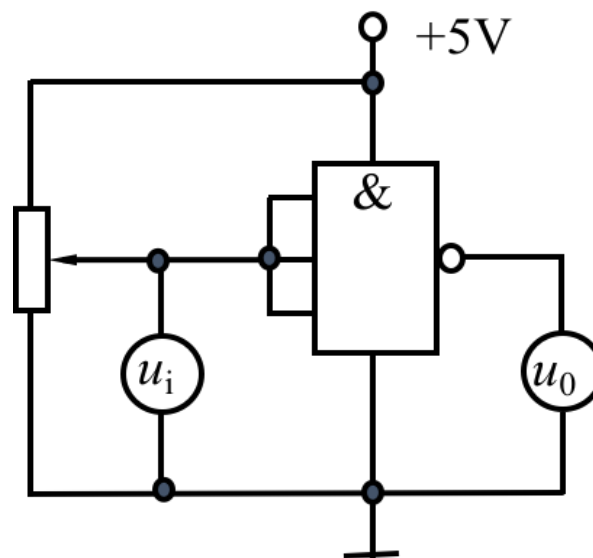
74LS00 的引脚排列图



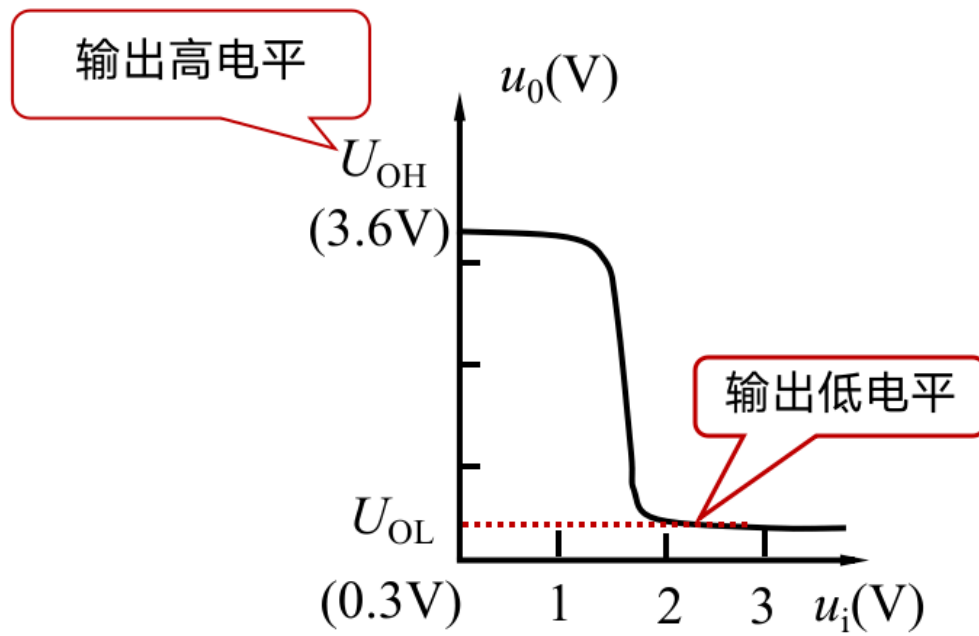
74LS20 的引脚排列图

TTL与非门的特性和技术参数

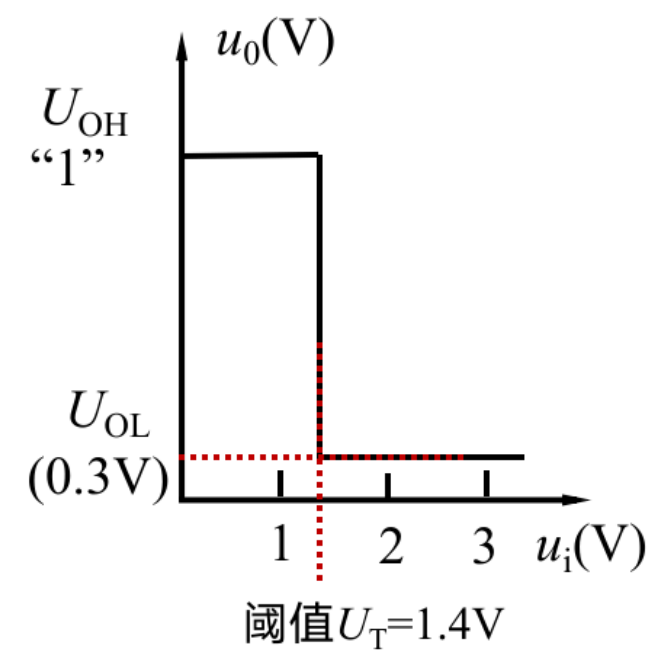
1. 电压传输特性



测试电路



传输特性曲线



理想的传输特性

2. 主要参数

(1) 输出高电平 U_{OH} 、输出低电平 U_{OL} $U_{OH} \geq 2.4V$ $U_{OL} \leq 0.4V$ 便认为合格。

典型值 $U_{OH}=3.6V$ $U_{OL} \leq 0.3V$

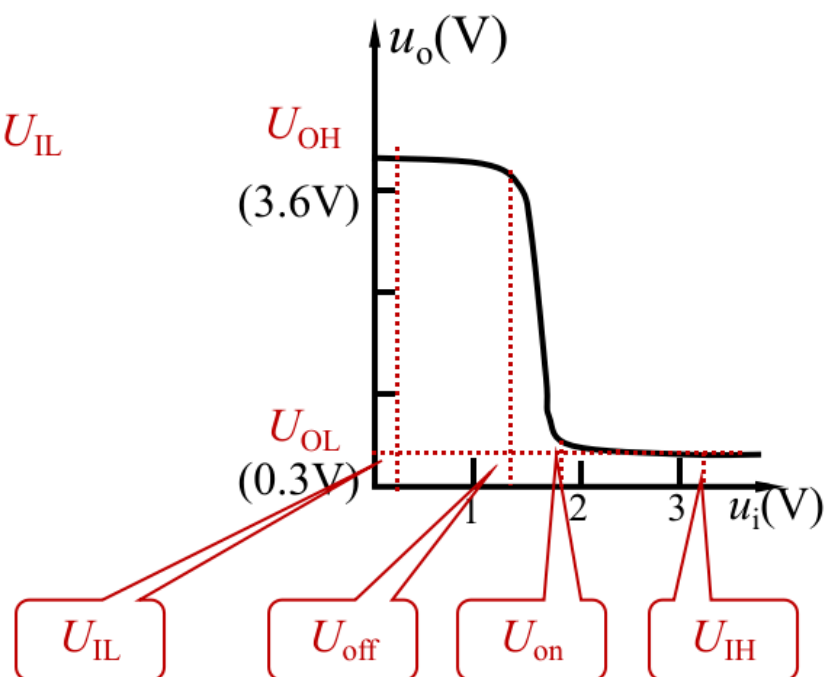
(2) 输入高电平 U_{IH} 、输入低电平 U_{IL}

典型值 $U_{IH}=3.6V$ $U_{IL} \leq 0.3V$

开门电平 $U_{on} = U_{IH}(\min)$

关门电平 $U_{off} = U_{IL}(\max)$

典型值 $U_{on}=1.8V$ $U_{off}=0.8V$

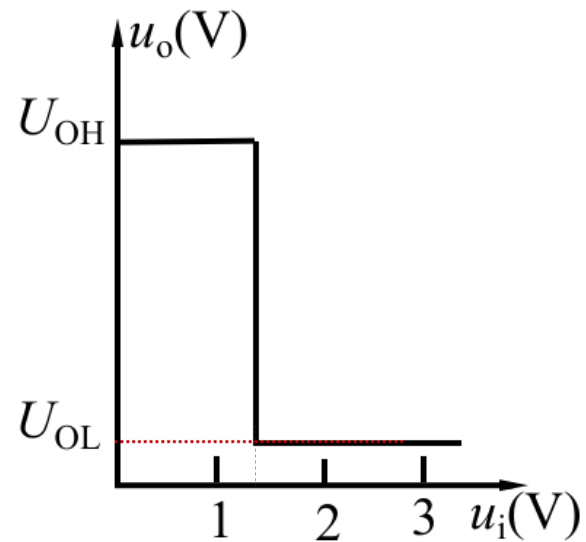


(3) 阈值电压 U_T

$u_i < U_T$ 时，认为 u_i 是低电平。

$u_i > U_T$ 时，认为 u_i 是高电平。

$$U_T = 1.4V$$



阈值 $U_T = 1.4V$

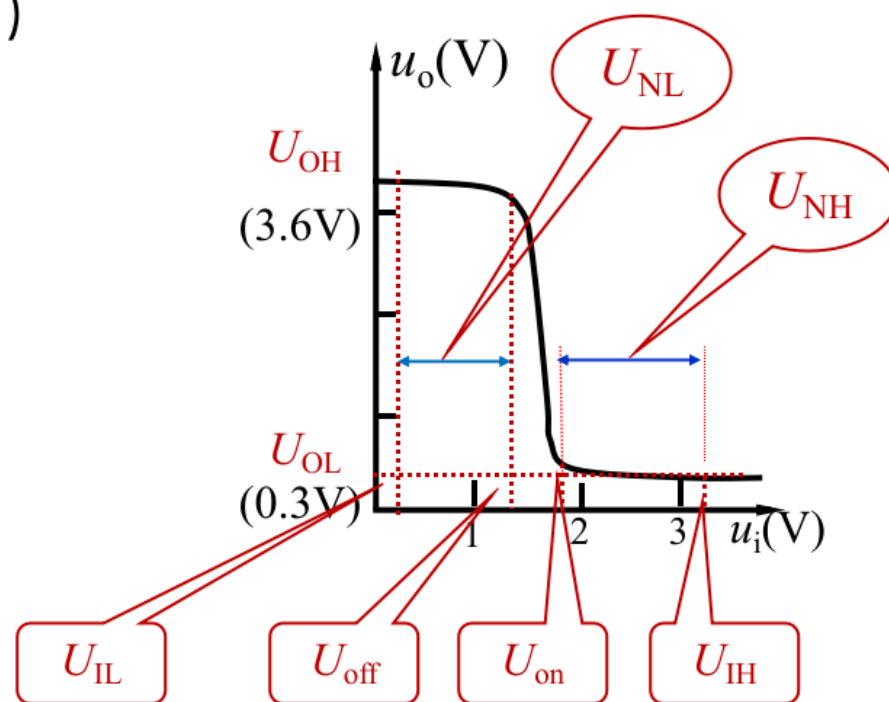
(4) 抗干扰容限（噪声容限）

低电平噪声容限：

$$U_{NL} = U_{off} - U_{IL}$$

高电平噪声容限：

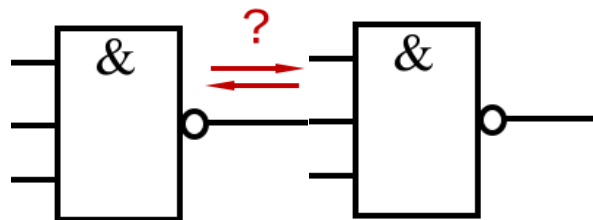
$$U_{NH} = U_{IH} - U_{on}$$



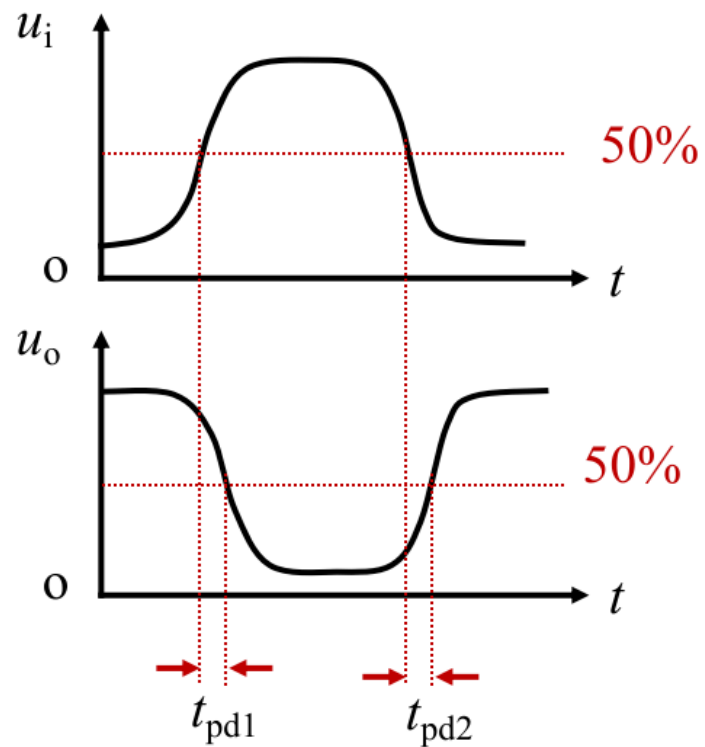
(5) 扇出系数

门电路输出驱动同类门的个数。

前后级之间电流的联系



(6) 平均传输时间



平均传输时间 $t_{pd} = \frac{1}{2}(t_{pd1} + t_{pd2})$

注意：

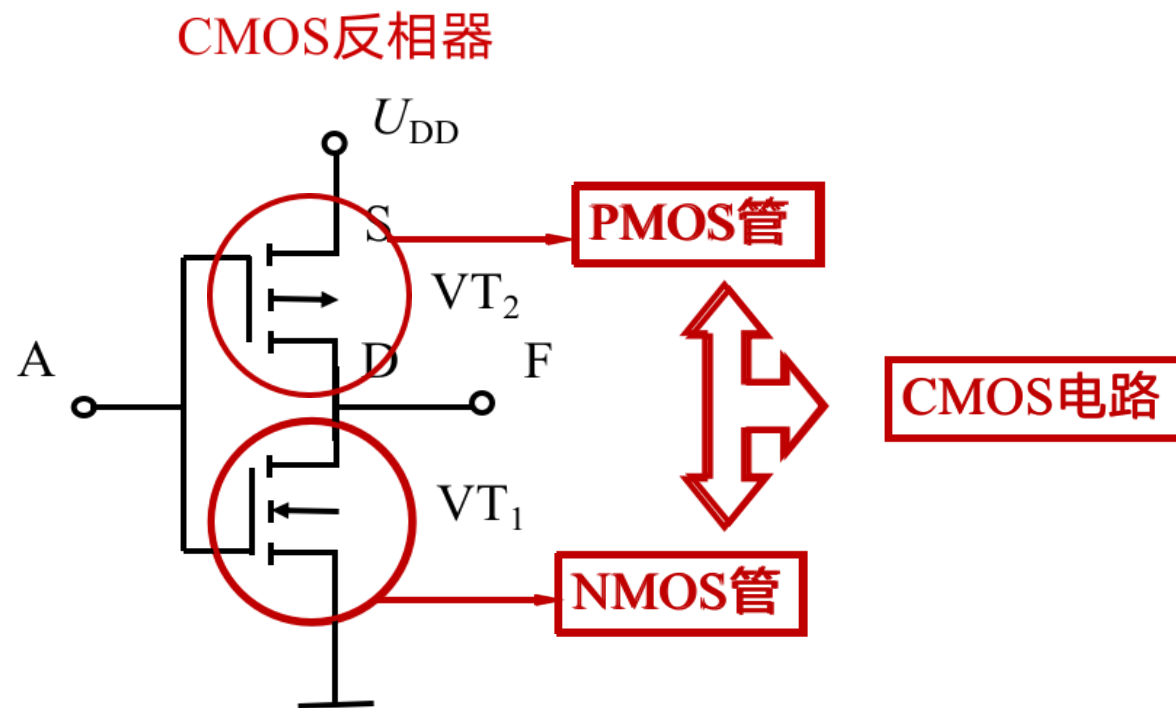
对于TTL电路来说

- 1、悬空的输入端相当于接高电平。
- 2、为了防止干扰，可将悬空的输入端接高电平。

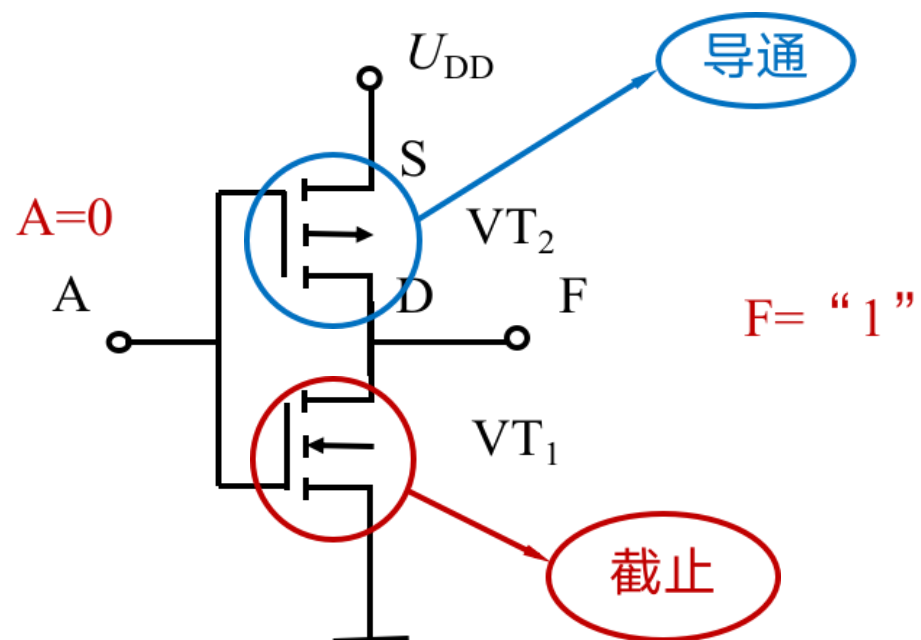


67

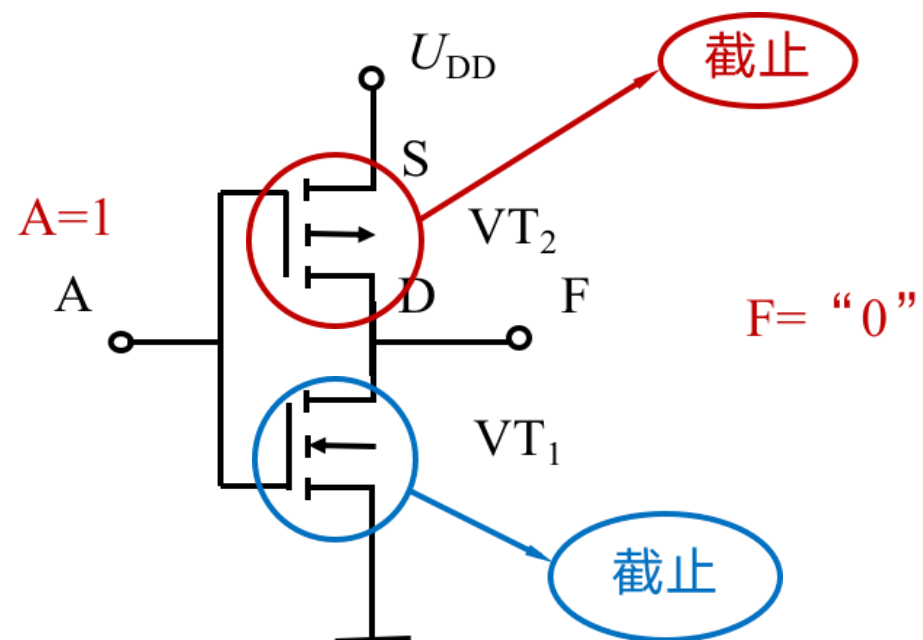
9.3.2 CMOS门电路



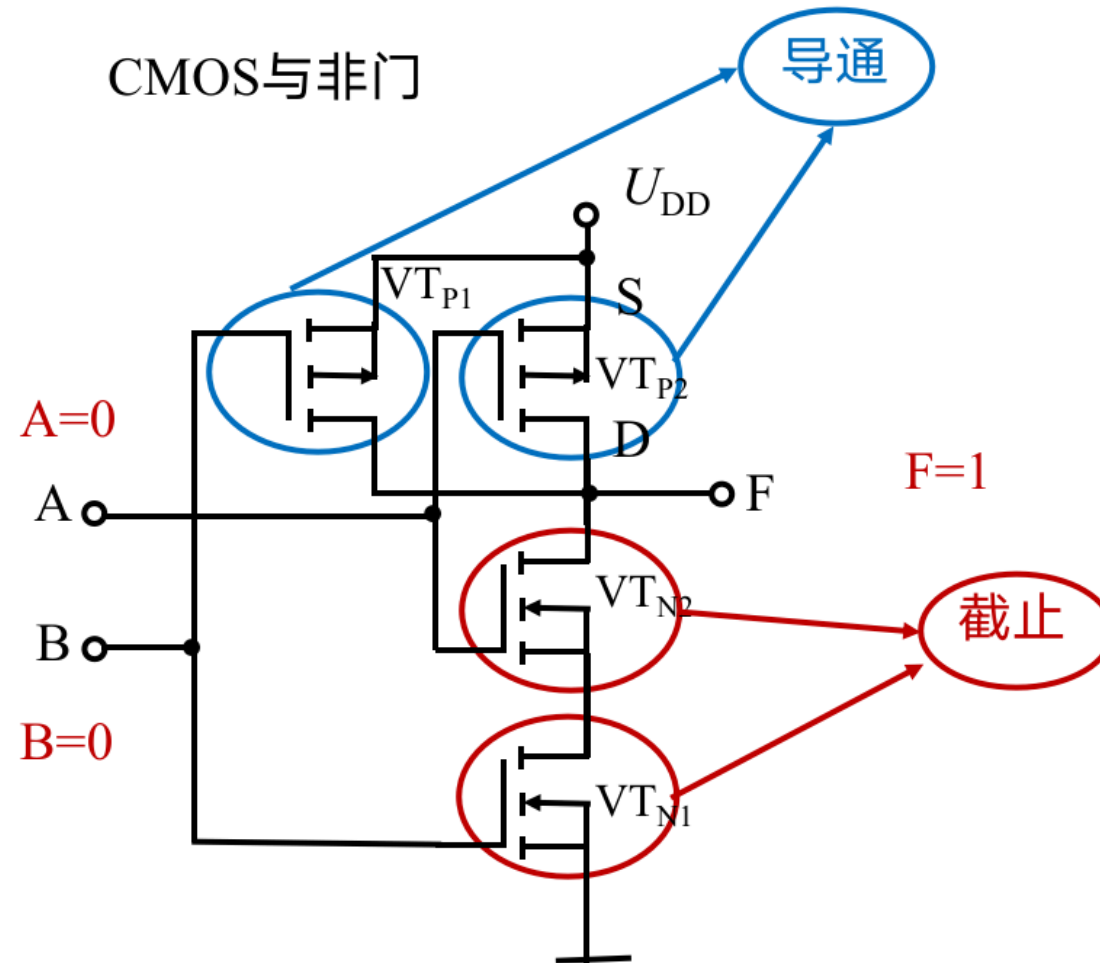
工作原理：



工作原理：

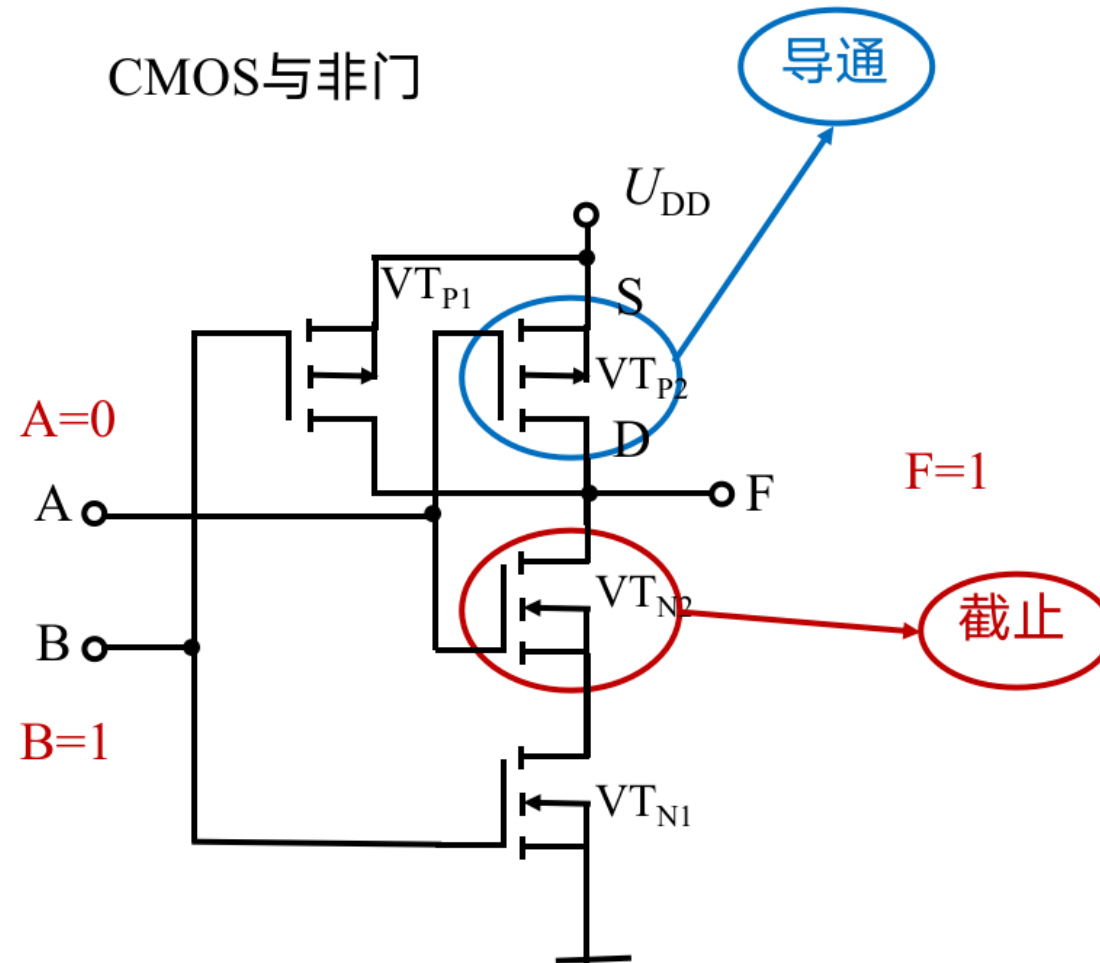


CMOS与非门



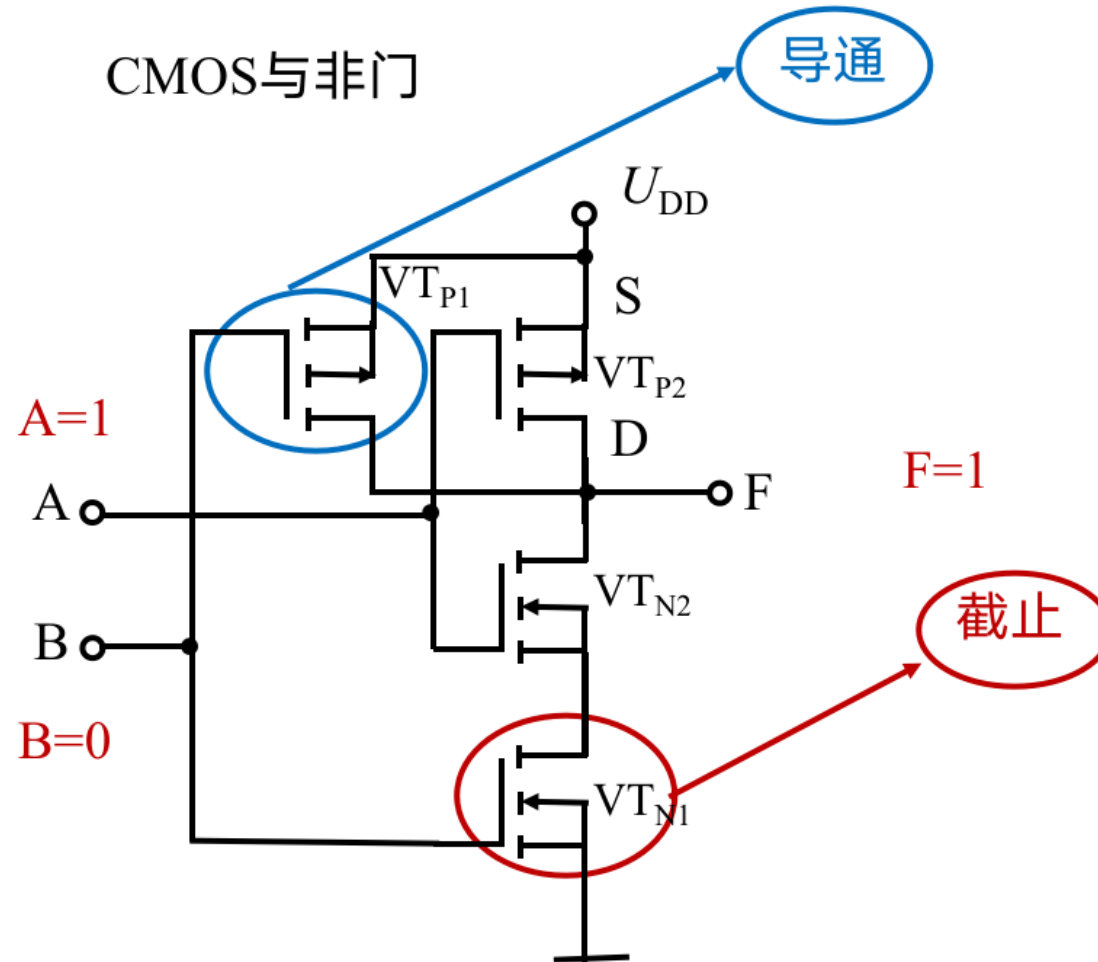
71

CMOS与非门



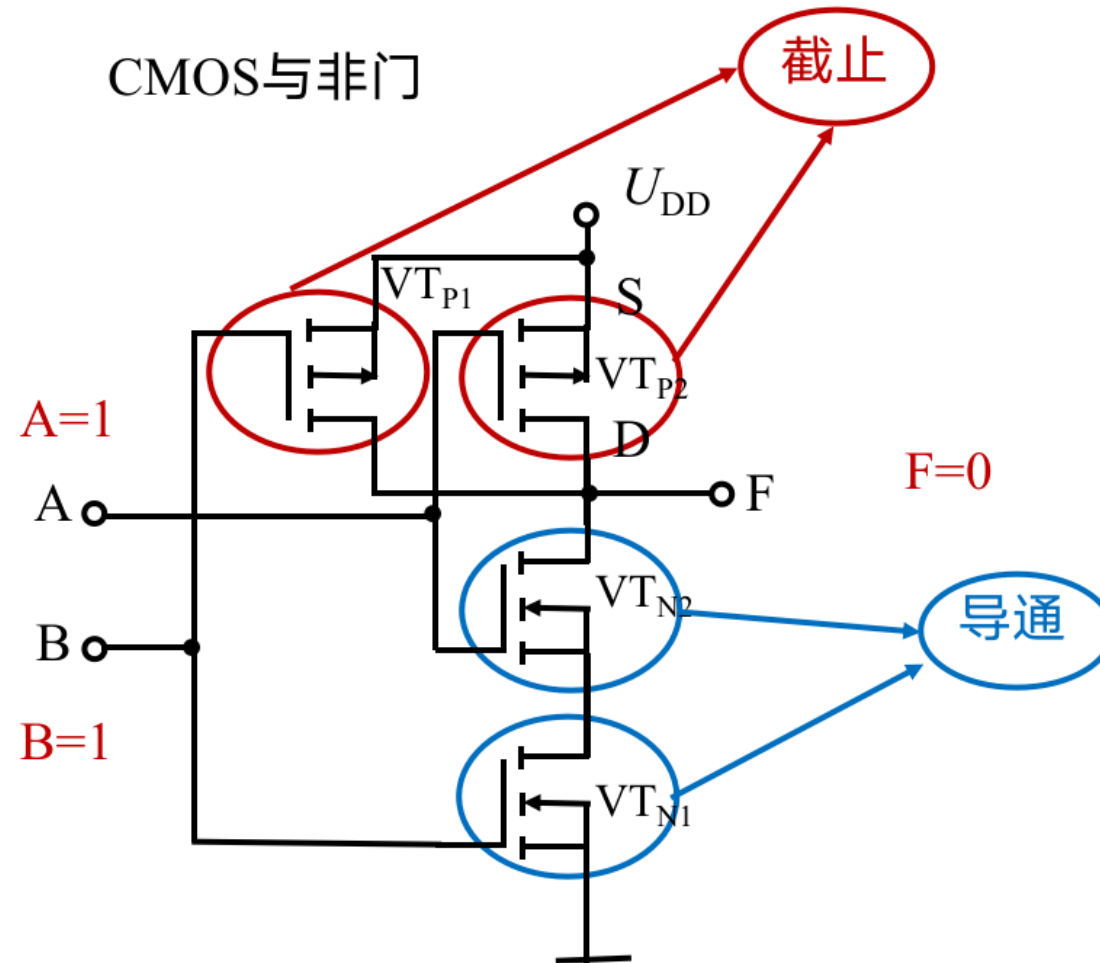
72

CMOS与非门



73

CMOS与非门



A	B	F
0	0	1
0	1	1
1	0	1
1	1	0

$$F = A \overline{B}$$

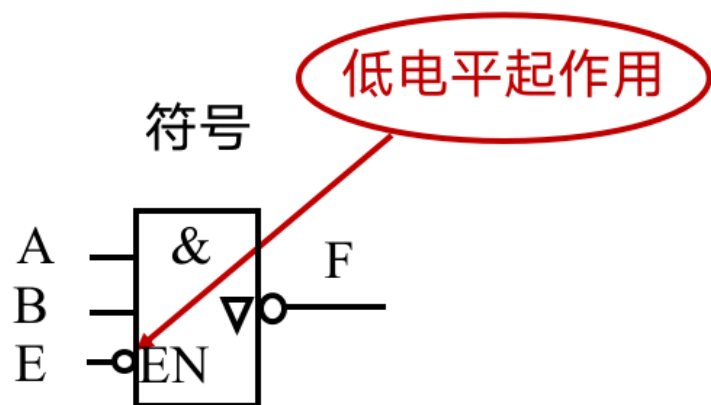
CMOS电路的优点：

- 1、静态功耗小。
- 2、允许电源电压范围宽（3~18V）。
- 3、扇出系数大，抗噪容限大。



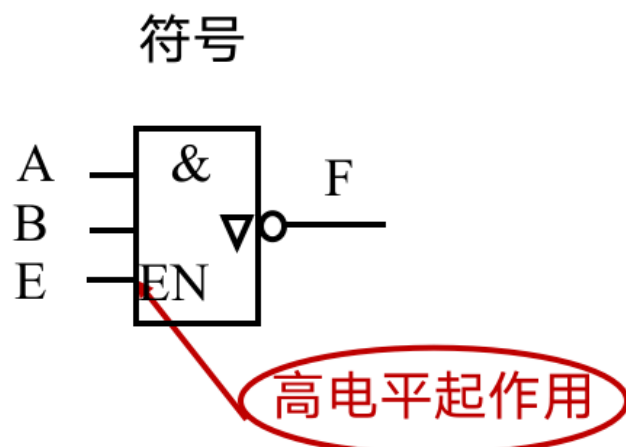
75

9.3.3 三态输出与非门电路



功能表

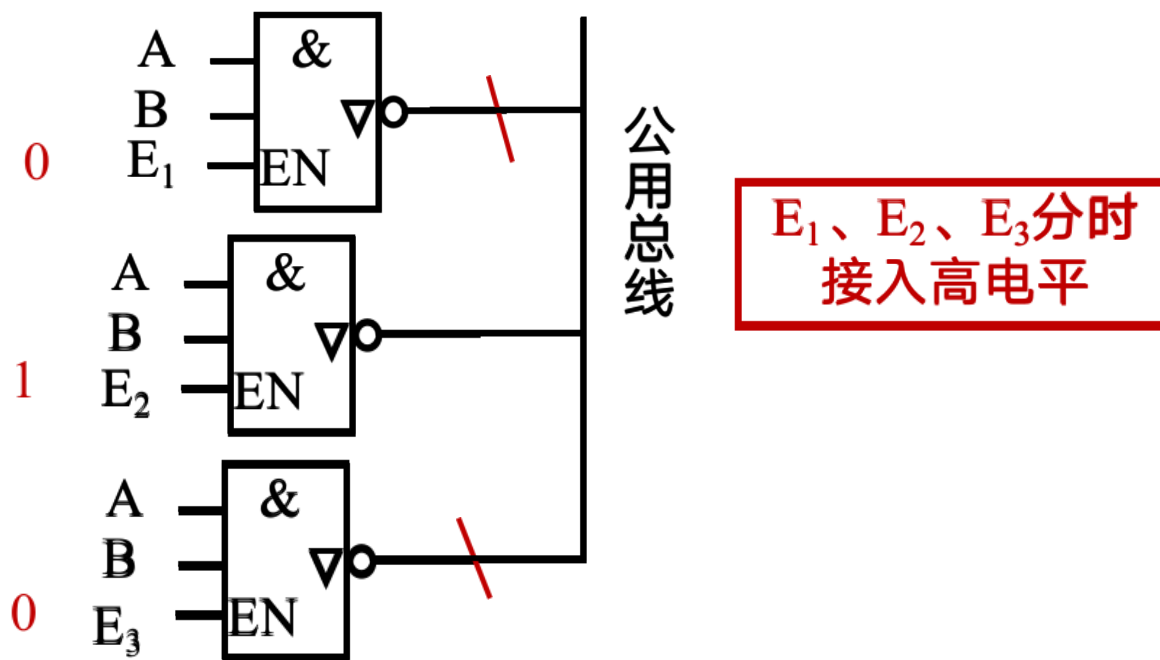
$E=0$	$F = \overline{AB}$
$E=1$	输出高阻



功能表

$E = 1$	$F = \overline{AB}$
$E = 0$	输出高阻

用途： 三态门主要作为TTL电路与**总线**间的接口电路。



77

9.3.4 集成逻辑门电路中的几个实际问题

1. TTL门电路与CMOS门电路性能的比较

(1) 与TTL相比，CMOS的功耗非常低。

(2) TTL电源电压范围较窄，典型值为+5V；

CMOS电源电压范围较宽(3-18V)，易于与其他电路接口。

(3) CMOS门电路的工作速度低于TTL门电路。

(4) TTL扇出数小，CMOS直流扇出能量强。

当前HCMOS可以兼顾速度和功耗

(5) TTL无静电敏感问题，CMOS容易被静电击穿。

早期，CMOS慢的原因是输入栅极电容较大，电容越大，频率越高，延迟越大，而TTL是双极性管，无大输入电容。

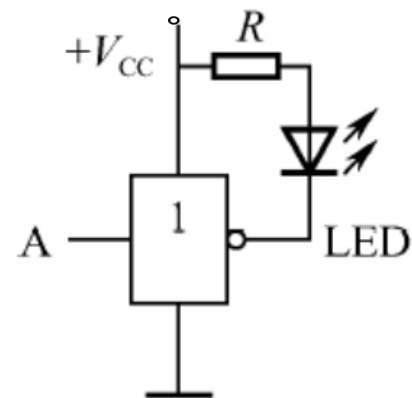
TTL延迟约8~12ns，而CMOS约50~100ns。

当前，随着工艺和集成度的提升，CMOS延迟已经接近甚至超越TTL，达到2-4ns。因此大规模集成电路，CPU，GPU，FPGA、SoC全靠微缩制程把CMOS速度推到GHz级别，远高于传统TTL。

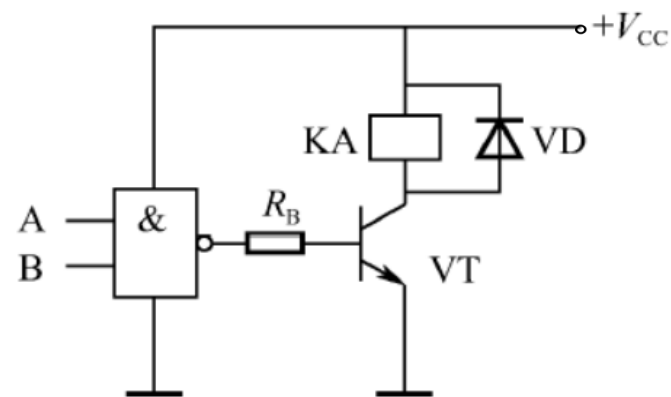
目前，纳米制程CMOS，速度碾压传统TTL，同时保留低功耗优势。

3. 门电路带负载时的接口电路

(1) 门电路驱动显示器件



(2) 门电路驱动机电性负载



80

4. 多余输入端的处理

TTL: $\begin{cases} \text{悬空 (易引入干扰)} \\ \text{直接接} +U_{CC} \\ \text{通过合适电阻接} +U_{CC} \end{cases}$ CMOS: $\begin{cases} \text{直接接} +U_{DD} \\ \text{不许悬空!} \end{cases}$

5. 输出端的连接: (三态门、OC门除外)

输出端不能并联!

输出端不能接地!

输出端不能直接接电源!



81

9.4 组合逻辑电路的分析与设计

9.4.1 组合逻辑电路的分析

9.4.2 组合逻辑电路的设计



82

逻辑
电路

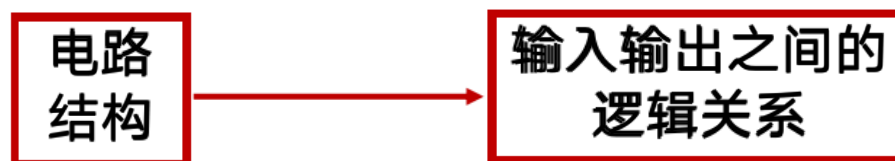
组合逻辑电路

当前的输出仅取决于当前的输入

时序逻辑电路

除与当前输入有关外还与原状态有关

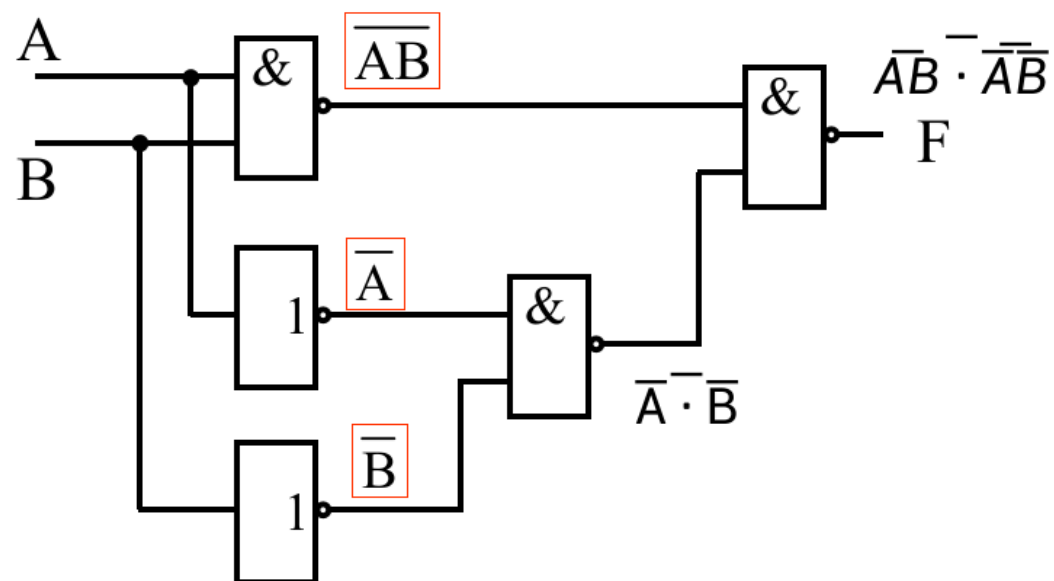
9.4.1 组合逻辑电路的分析



分析步骤:

- 1、由给定的逻辑图写出逻辑关系表达式;
- 2、运用逻辑代数对逻辑式进行化简或变换;
- 3、列出输入输出状态表并得出结论;
- 4、根据输入输出状态表判断逻辑功能。

例11分析下图的逻辑功能。



$$F = \overline{AB} \cdot \overline{A}\overline{B} = \overline{A}\overline{B} + \overline{A}\overline{B} = \overline{A}\overline{B} + \overline{A}\overline{B}$$

$$F = \overline{A}B + A\overline{B} = \overline{A}B + A\overline{B} \equiv A \oplus B$$

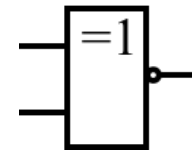
状态表

A	B	F
0	0	1
0	1	0
1	0	0
1	1	1

相同为“1”

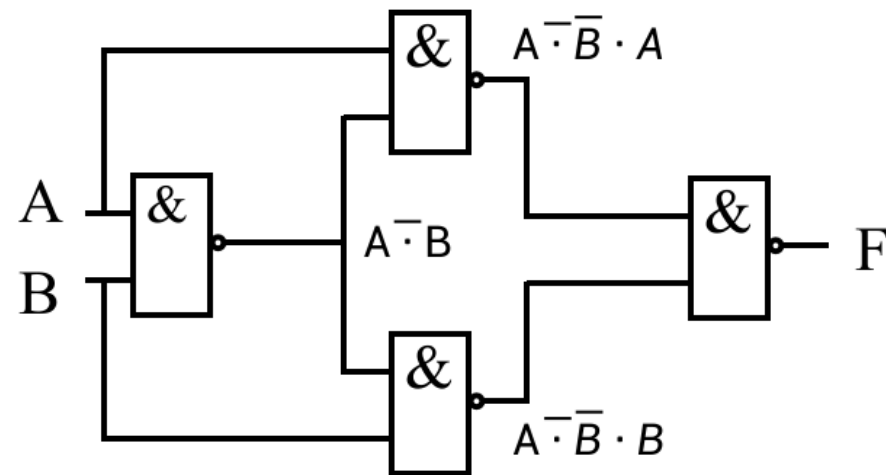
不同为“0”

同或门



$$F = \overline{A \oplus B}$$

例12分析下图的逻辑功能。



$$\begin{aligned} F &= A \cdot \bar{B} \cdot A + A \cdot \bar{B} \cdot B \\ &= A \cdot \bar{B} \cdot A + A \cdot \bar{B} \cdot B \\ &= (\bar{A} + \bar{B}) \cdot A + (\bar{A} + \bar{B}) \cdot B = A \cdot \bar{B} + \bar{A} \cdot B \end{aligned}$$

87

$$F = A \cdot \bar{B} + \bar{A} \cdot B$$

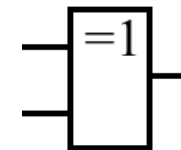
状态表

A	B	F
0	0	0
0	1	1
1	0	1
1	1	0

相同为“0”

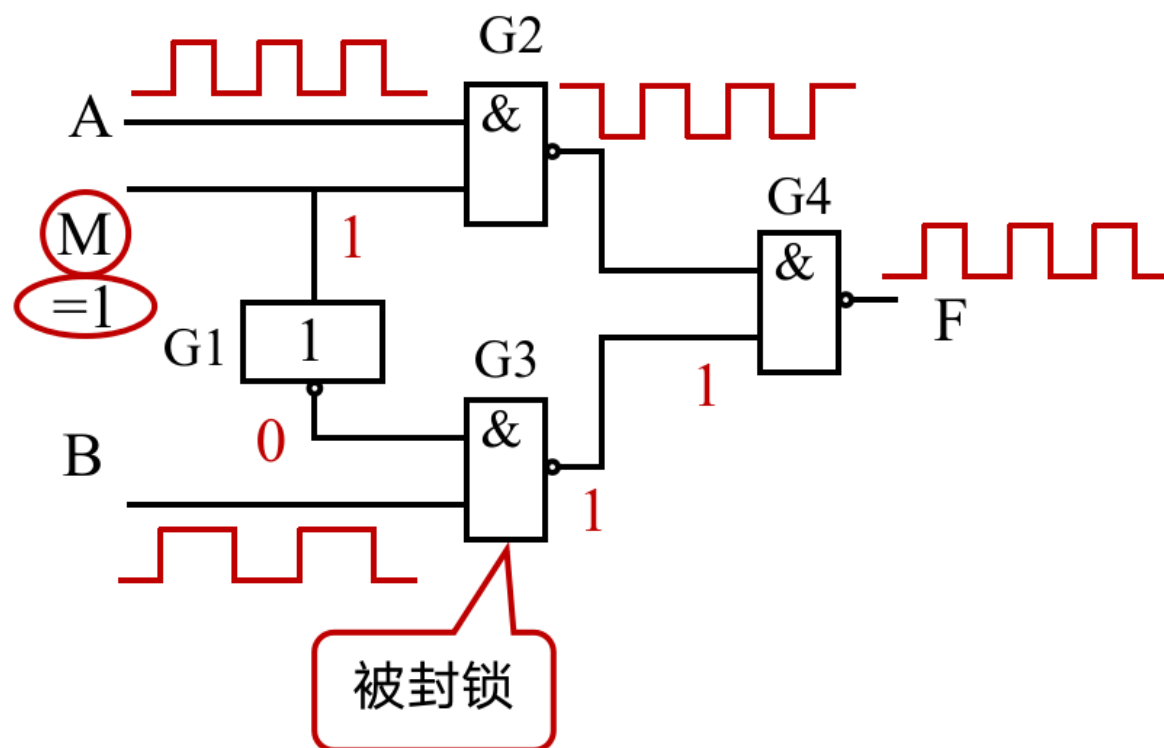
不同为“1”

异或门

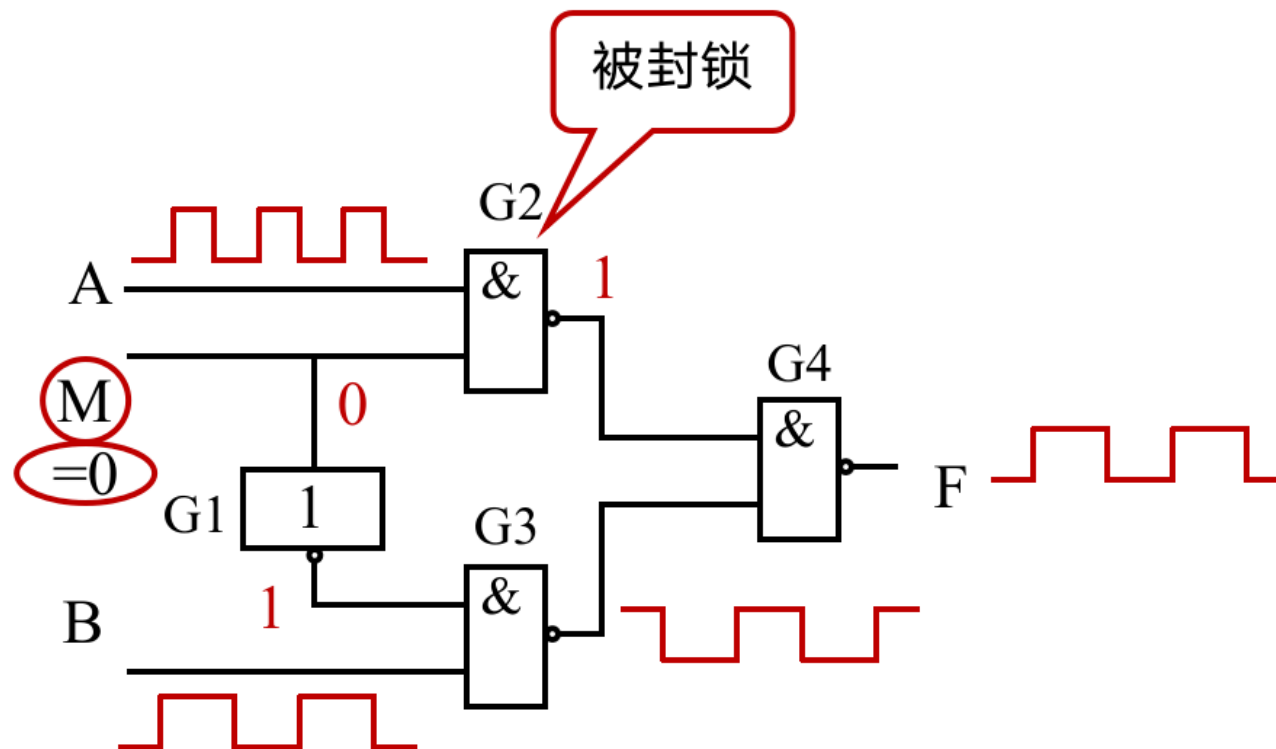


$$F = A \oplus B$$

例13分析下图的逻辑功能。



89



90

9.4.2 组合逻辑电路的设计

任务
要求

最简单的
逻辑电路

分析步骤：

- 1、指定实际问题的逻辑含义，列出逻辑状态表；
- 2、根据状态表，写出逻辑式；
- 3、用逻辑代数对逻辑式进行化简或变换；
- 4、根据化简、变换后的逻辑式画出逻辑图。

91

例14 设计三人表决电路（A、B、C）。每人一个按键，如果同意则按下，不同意则不按。结果用指示灯表示，多数同意时指示灯亮，否则不亮。

(1) 首先指明逻辑符号取“0”、“1”的含义。

三个按键A、B、C：按下时为“1”，不按时为“0”。
输出是F：多数赞成时是“1”，否则是“0”。

(2) 根据题意列出逻辑状态表。

逻辑状态表

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

(3) 根据状态表，写出逻辑式。

①取结果为“1”的项，将输入变量写成“与”的形式；（变量为1，取原变量，变量为0，取反变量）

$$F = \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC$$

②将各项写成“或”的形式

(4) 化简逻辑式。

代数化简

$$F = \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC$$

$$= \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC + \textcolor{red}{ABC} + \textcolor{red}{ABC}$$

$$= \bar{A}BC + \textcolor{red}{ABC} + A\bar{B}C + \textcolor{red}{ABC} + AB\bar{C} + ABC$$

$$= (\bar{A}+A)BC + A(\bar{B}+B)C + AB(\bar{C}+C)$$

$$= BC + AC + AB$$

$$F = AB + BC + AC$$

卡诺图化简

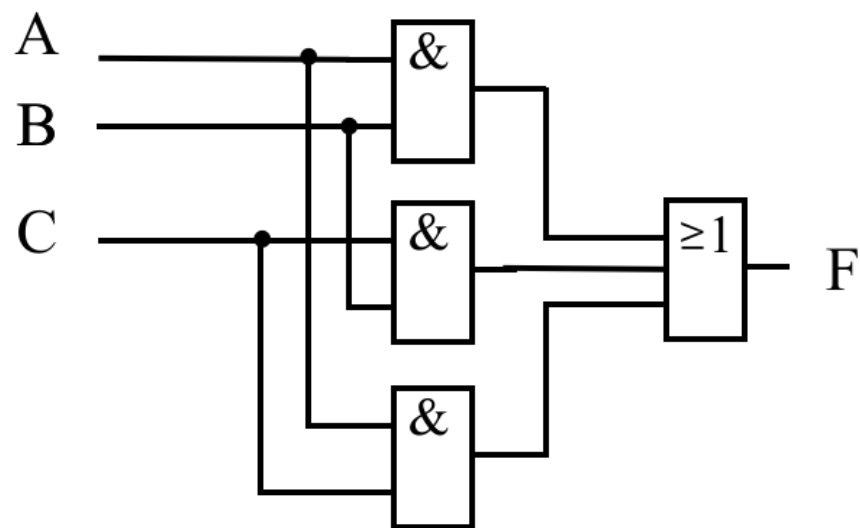
$$F = \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC$$

BC A	BC			
	00	01	11	10
0			1	
1		1	1	1

$$F = AC + AB + BC$$

(5) 根据逻辑表达式画出逻辑图。

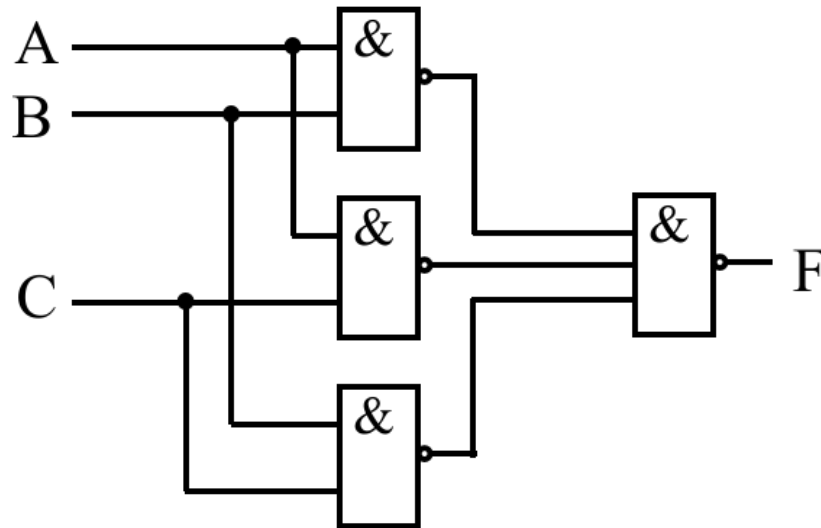
$$F = AB + BC + AC$$



若用与非门实现

$$F = AB + BC + AC$$

$$= \overline{\overline{AB + BC + AC}} = \overline{AB} \cdot \overline{BC} \cdot \overline{CA}$$



97

9.5 常用组合逻辑模块

9.5.1 加法器

9.5.2 编码器

9.5.3 译码器

9.5.4 数据分配器与数据选择器

9.5.5 用中规模组合逻辑模块实现组合逻辑函数



98

9.5.1 加法器

1. 半加器

半加运算不考虑从低位来的进位

A---加数； B---被加数； S---本位和； C---进位。

逻辑状态表

A	B	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

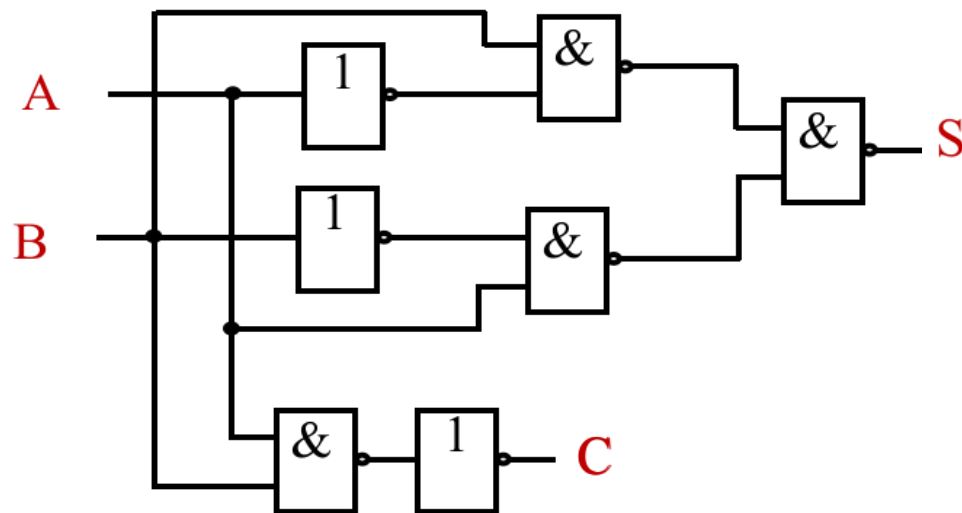
$$S = \bar{A}B + A\bar{B} = A \oplus B \quad C = AB$$

用与非门实现

$$S = \bar{A}B + A\bar{B} = \overline{\bar{A}B} \cdot \overline{A\bar{B}} = \overline{\bar{A}B} \cdot \overline{A\bar{B}}$$

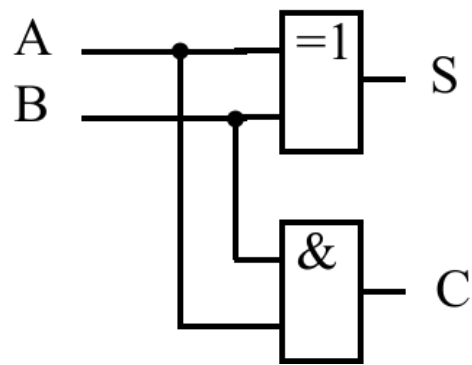
$$C = \overline{\bar{A}B}$$

画出逻辑图 由逻辑式知，需七个门

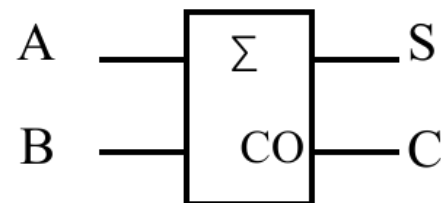


100

用异或门构成



逻辑符号



2. 全加器 全加运算考虑从低位来的进位

A_n ---加数； B_n ---被加数； C_{n-1} ---低位的进位；

S_n ---本位和； C_n ---进位。

逻辑状态表

被加数 A_n	加数 B_n	低位进位 C_{n-1}	本位和 S_n	本位进位 C_n
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$S_n = \bar{A}_n \bar{B}_n C_{n-1} + \bar{A}_n B_n \bar{C}_{n-1} + A_n \bar{B}_n \bar{C}_{n-1} + A_n B_n C_{n-1}$$

$$C_n = \bar{A}_n B_n C_{n-1} + A_n \bar{B}_n C_{n-1} + A_n B_n \bar{C}_{n-1} + A_n B_n C_{n-1}$$

102

$$S_n = \underline{\bar{A}_n \bar{B}_n C_{n-1}} + \underline{\bar{A}_n B_n \bar{C}_{n-1}} + \underline{A_n \bar{B}_n \bar{C}_{n-1}} + \underline{A_n B_n C_{n-1}}$$

$$C_n = \underline{\bar{A}_n B_n C_{n-1}} + \underline{A_n \bar{B}_n C_{n-1}} + \underline{A_n B_n \bar{C}_{n-1}} + \underline{A_n B_n C_{n-1}}$$

$$\begin{aligned} S_n &= (\bar{A}_n \bar{B}_n + A_n B_n) C_{n-1} + (\bar{A}_n B_n + A_n \bar{B}_n) \bar{C}_{n-1} \\ &= (A_n \odot B_n) C_{n-1} + ((A_n \oplus B_n) \bar{C}_{n-1}) = \overline{(A_n \oplus B_n)} C_{n-1} + (A_n \oplus B_n) \bar{C}_{n-1} \end{aligned}$$

$$S_n = (A_n \oplus B_n) \oplus C_{n-1}$$

$$C_n = (\bar{A}_n B_n + A_n \bar{B}_n) C_{n-1} + A_n B_n (\bar{C}_{n-1} + C_{n-1})$$

$$= (\bar{A}_n B_n + A_n \bar{B}_n) C_{n-1} + A_n B_n$$

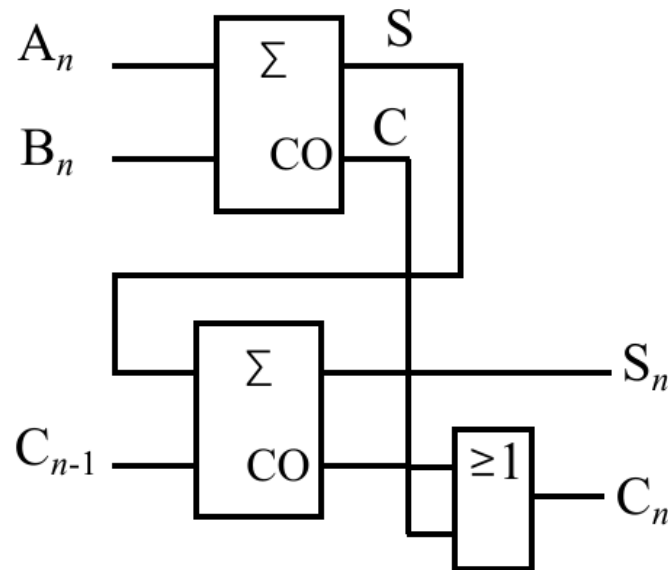
$$C_n = (A_n \oplus B_n) C_{n-1} + A_n B_n$$

全加器输出 $S_n = (A_n \oplus B_n) \oplus C_{n-1} = S \oplus C_{n-1}$

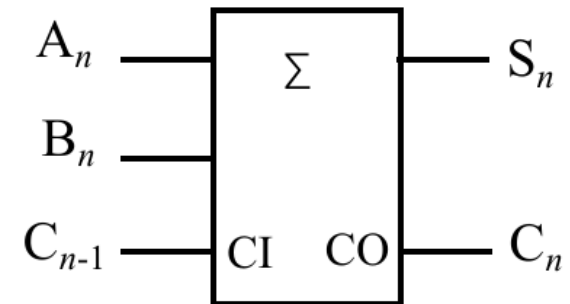
$$C_n = (A_n \oplus B_n)C_{n-1} + A_n B_n = SC_{n-1} + A_n B_n$$

半加器输出 $S = A \oplus B$ $C = AB$

全加器逻辑图

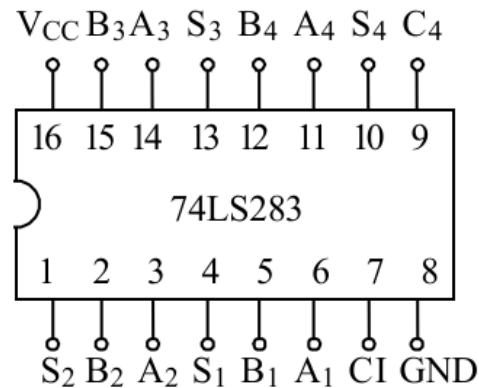


逻辑符号

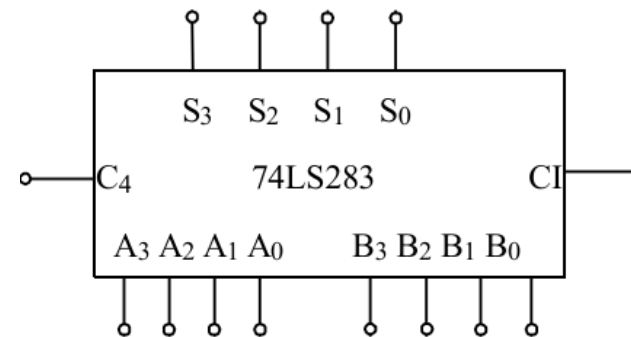


104

4位超前进位TTL加法器74LS283



(a) 引脚排列图



(b) 逻辑功能示意图

两组4位二进制数输入： $A_4A_3A_2A_1$ 、 $B_4B_3B_2B_1$ ；

一个低位向本位的进位输入： CI ；

一组二进制数输出： $S_4S_3S_2S_1$ ；

一个最高位的进位输出： C_4 。

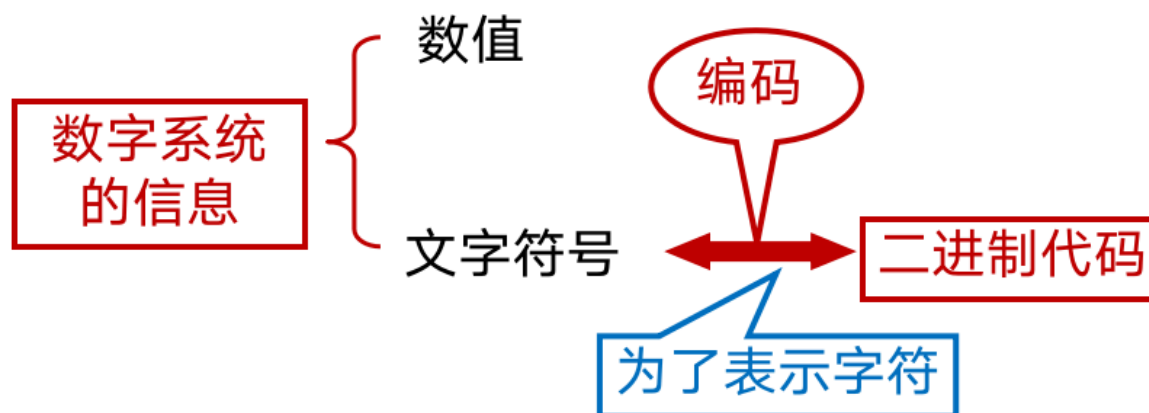
$$\begin{array}{r}
 A_4 \ A_3 \ A_2 \ A_1 \\
 B_4 \ B_3 \ B_2 \ B_1 \\
 + \quad \quad \quad CI \\
 \hline
 C_4 \ S_4 \ S_3 \ S_2 \ S_1
 \end{array}$$



9.5.2 编码器

编码：赋予选定的一系列二进制代码以固定的含义。

编码器：实现编码功能的逻辑电路。



1. 二进制编码器

将一系列信号状态编制成二进制代码。

n 个二进制代码（ n 位二进制数）有 2^n 种不同的组合，
可以表示 2^n 个信号。

输入： N 个信号
输出： n 位二进制代码

} $2^n \geq N$

例15 用与非门组成三位二进制编码器。---八线-三线编码器

输入： $I_1 \sim I_8$ 八个信号

输出： Y_2 、 Y_1 、 Y_0 三位二进制数

因为： $2^3 = 8$

- 组合逻辑电路
设计步骤
- 1、列出状态表（编码表）
 - 2、写出逻辑表达式并进行化简和变换
 - 3、根据化简和变换后的逻辑式画出逻辑图

编码表

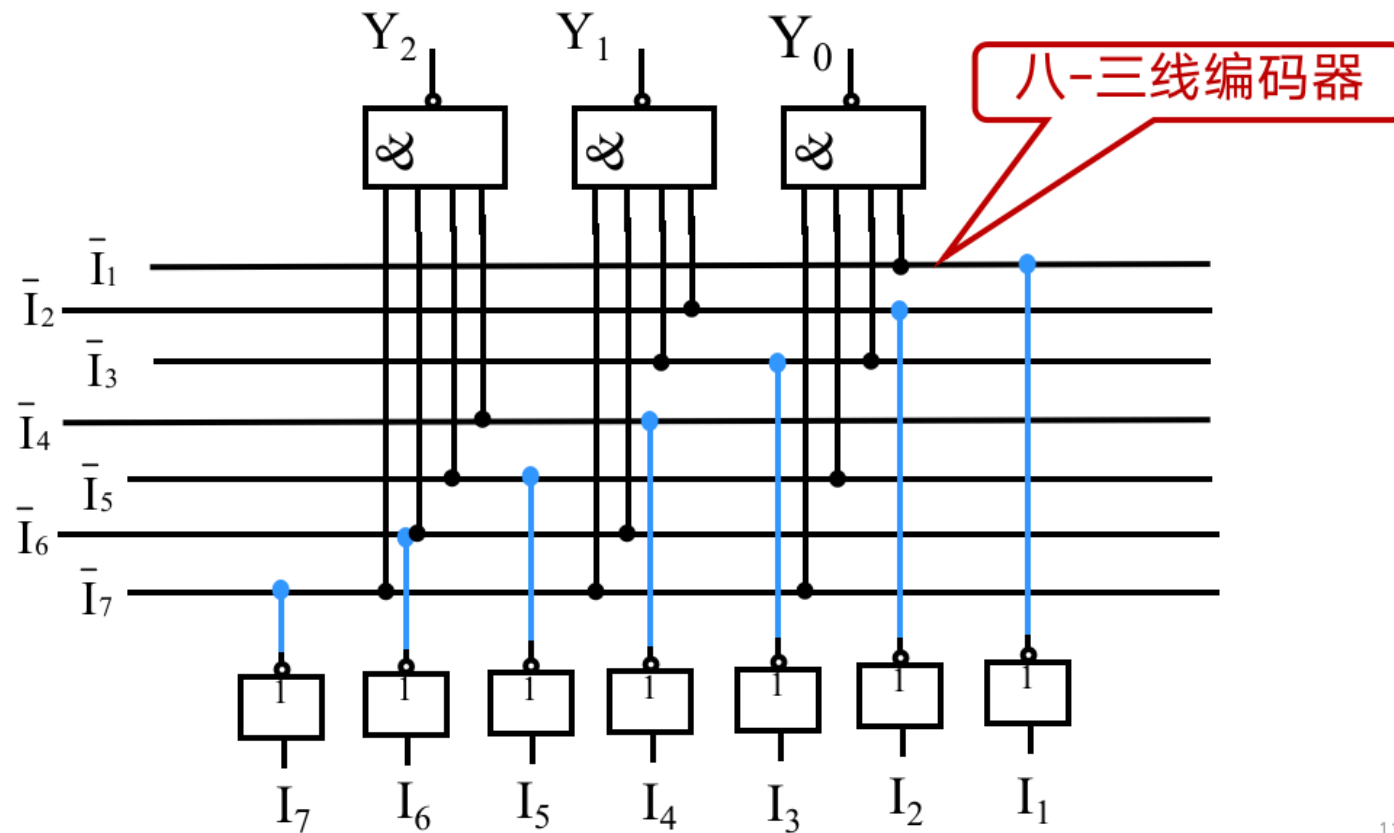
I ₀	I ₁	I ₂	I ₃	I ₄	I ₅	I ₆	I ₇	Y ₂	Y ₁	Y ₀
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

$$Y_0 = I_1 + I_3 + I_5 + I_7 = \overline{I_1} \cdot \overline{I_3} \cdot \overline{I_5} \cdot \overline{I_7}$$

$$Y_1 = \overline{I_2} \cdot \overline{I_3} \cdot \overline{I_6} \cdot \overline{I_7} \quad Y_2 = \overline{I_4} \cdot \overline{I_5} \cdot \overline{I_6} \cdot \overline{I_7}$$

109

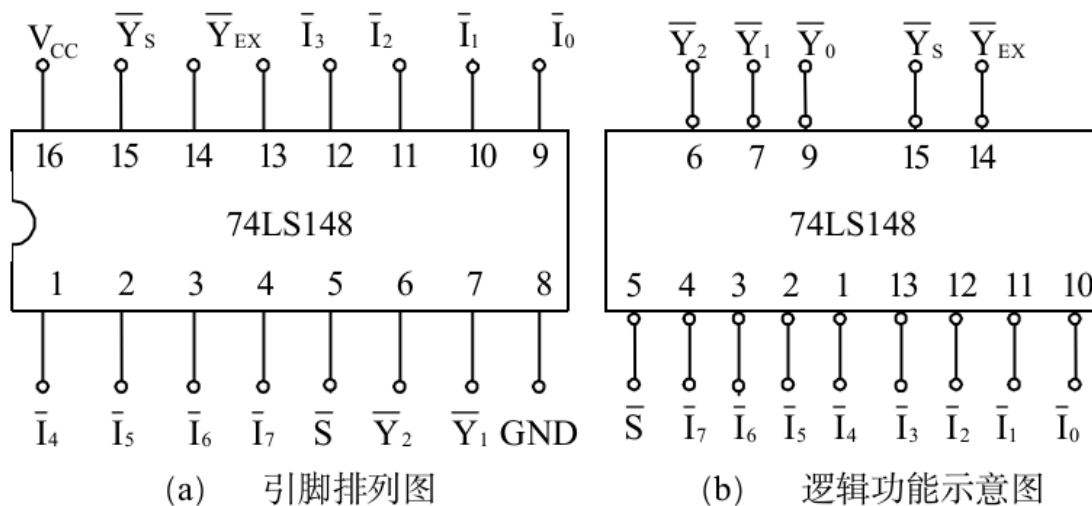
$$Y_0 = \overline{\bar{I}_1} \cdot \overline{\bar{I}_3} \cdot \overline{\bar{I}_5} \cdot \overline{\bar{I}_7} \quad Y_1 = \overline{\bar{I}_2} \cdot \overline{\bar{I}_3} \cdot \overline{\bar{I}_6} \cdot \overline{\bar{I}_7} \quad Y_2 = \overline{\bar{I}_4} \cdot \overline{\bar{I}_5} \cdot \overline{\bar{I}_6} \cdot \overline{\bar{I}_7}$$



8-3线优先编码器

当待编码的几个信号同时为1时，应按照事先编排好的优先顺序输出。具有此种功能的编码器为**优先编码器**。

74LS148为8-3线优先编码器



$\overline{S}$ 为使能输入端，低电平有效；

$\overline{Y}_S$ 为选通输出端，
与使能输入端配合可以实现
多级编码器之间的优先级别
的控制。

$\overline{Y}_{EX}$ 为扩展输出端，
0表示是编码输出
1表示不是编码输出

111

74LS148编码器状态表

输入：逻辑0(低电平) 有效

输出：逻辑0(低电平) 有效

输 入									输 出				
$\bar{S}$	$\bar{I}_0$	$\bar{I}_1$	$\bar{I}_2$	$\bar{I}_3$	$\bar{I}_4$	$\bar{I}_5$	$\bar{I}_6$	$\bar{I}_7$	$\bar{Y}_2$	$\bar{Y}_1$	$\bar{Y}_0$	$\bar{Y}_{EX}$	$\bar{Y}_S$
0	0	1	1	1	1	1	1	1	1	1	1	0	1
0	×	0	1	1	1	1	1	1	1	1	0	0	1
0	×	×	0	1	1	1	1	1	1	0	1	0	1
0	×	×	×	0	1	1	1	1	1	0	0	0	1
0	×	×	×	×	0	1	1	1	0	1	1	0	1
0	×	×	×	×	×	0	1	1	0	1	0	0	1
0	×	×	×	×	×	×	0	1	0	0	1	0	1
0	×	×	×	×	×	×	×	0	0	0	0	0	1
0	1	1	1	1	1	1	1	1	1	1	1	1	0
1	×	×	×	×	×	×	×	×	1	1	1	1	1

112

2. 二进制十进制编码器

二进制十进制 (BCD码)

二进制编码的十进制数
(Binary Coded Decimal Codes)

最常用的BCD码：8421码

在BCD码中，用四位二进制数表示0~9十个数码。

十进制数 $(N)_D$ 与二进制编码 $(K_3K_2K_1K_0)_B$ 关系可以表示为：

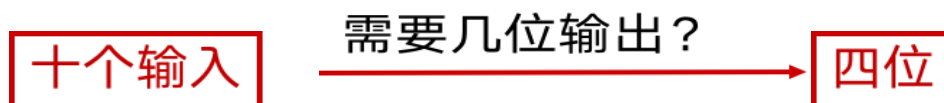
$$(N)_D = 2^3 K_3 + 2^2 K_2 + 2^1 K_1 + 2^0 K_0$$



BCD码				十进制数
0	0	0	0	0
0	0	0	1	1
0	0	1	0	2
0	0	1	1	3
0	1	0	0	4
0	1	0	1	5
0	1	1	0	6
0	1	1	1	7
1	0	0	0	8
1	0	0	1	9
“8” “4” “2” “1”				

二---十进制编码器

将十个状态（对应于十进制的十个代码）编制成BCD码。



输入: $I_0 \sim I_9$

输出: $Y_0 \sim Y_3$



列出状态表如下:

编码表

输入	Y_3	Y_2	Y_1	Y_0
I_0	0	0	0	0
I_1	0	0	0	1
I_2	0	0	1	0
I_3	0	0	1	1
I_4	0	1	0	0
I_5	0	1	0	1
I_6	0	1	1	0
I_7	0	1	1	1
I_8	1	0	0	0
I_9	1	0	0	1

逻辑式

$$Y_3 = I_8 + I_9 = \overline{\overline{I_8} \cdot \overline{I_9}}$$

$$Y_2 = \overline{\overline{I_4} \cdot \overline{I_5} \cdot \overline{I_6} \cdot \overline{I_7}}$$

$$Y_1 = \overline{\overline{I_2} \cdot \overline{I_3} \cdot \overline{I_6} \cdot \overline{I_7}}$$

$$Y_0 = \overline{\overline{I_1} \cdot \overline{I_3} \cdot \overline{I_5} \cdot \overline{I_7} \cdot \overline{I_9}}$$

逻辑图（略）



115

9.5.3 译码器

译码 编码的逆过程，即将某二进制代码翻译成电路的某种状态。

1. 二进制译码器

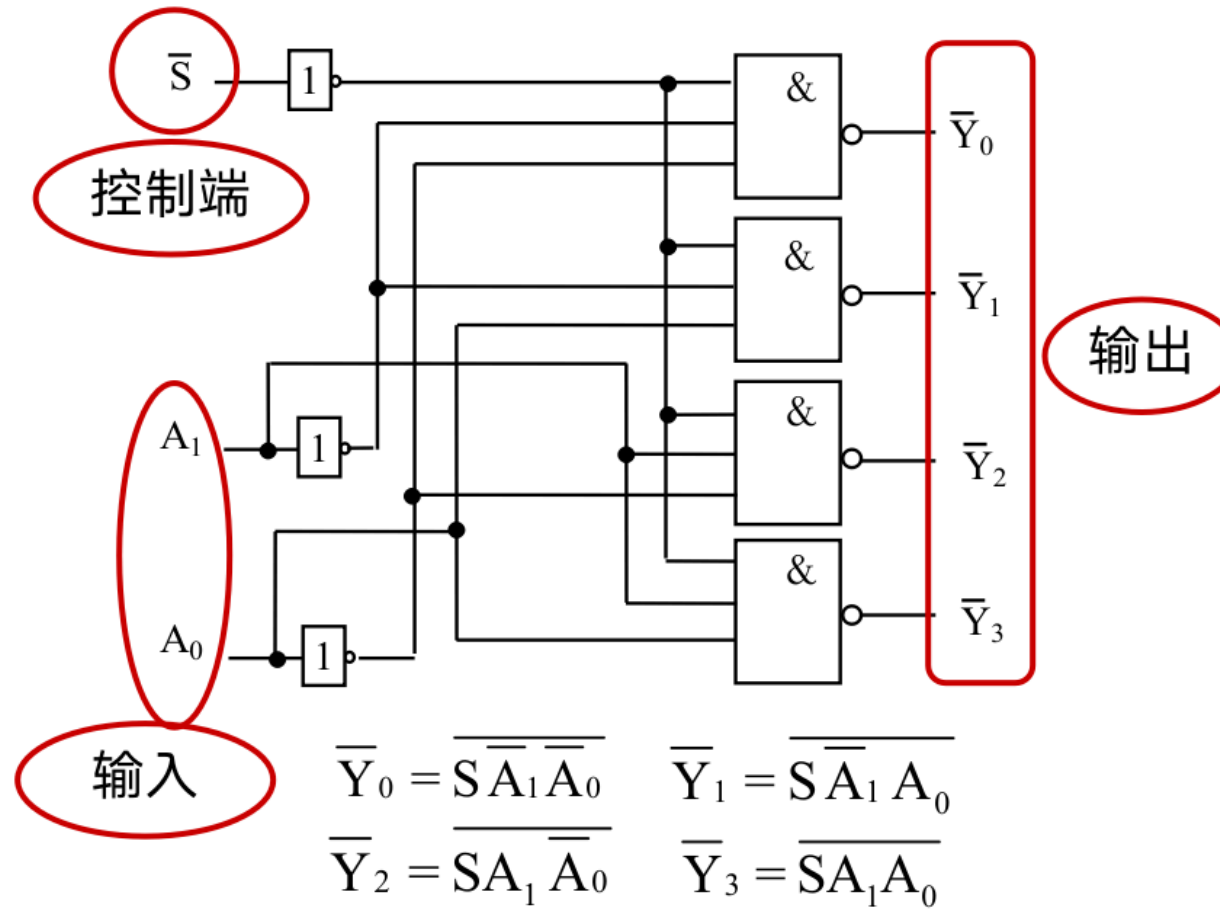
将输入的一组 n 位二进制码译成 2^n 种电路状态。

也叫 n --- 2^n 线译码器。

译码器的输入： n 位二进制代码

译码器的输出： 2^n 个高低电平信号

2-4线译码器的逻辑图



117

$$\overline{Y}_0 = \overline{S} \overline{A_1} \overline{A_0} \quad \overline{Y}_1 = \overline{S} \overline{A_1} A_0 \quad \overline{Y}_2 = \overline{S} A_1 \overline{A_0} \quad \overline{Y}_3 = \overline{S} A_1 A_0$$

由逻辑式得真值表

$\overline{S} = 0$ ，即 $S=1$ 时

$$\overline{Y}_0 = \overline{\overline{A_1} \overline{A_0}} = \overline{m_0}$$

$$\overline{Y}_1 = \overline{\overline{A_1} A_0} = \overline{m_1}$$

$$\overline{Y}_2 = \overline{A_1 \overline{A_0}} = \overline{m_2}$$

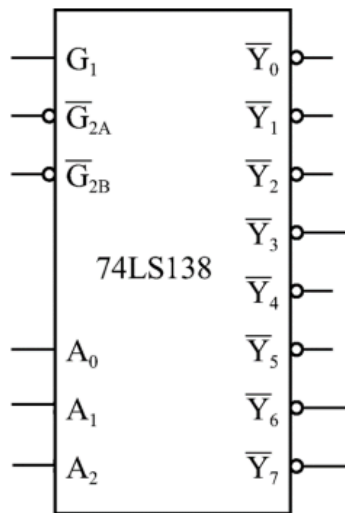
$$\overline{Y}_3 = \overline{A_1 A_0} = \overline{m_3}$$

$\overline{S}$	A_1	A_0	$\overline{Y}_0$	$\overline{Y}_1$	$\overline{Y}_2$	$\overline{Y}_3$
1	×	×	1	1	1	1
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	0

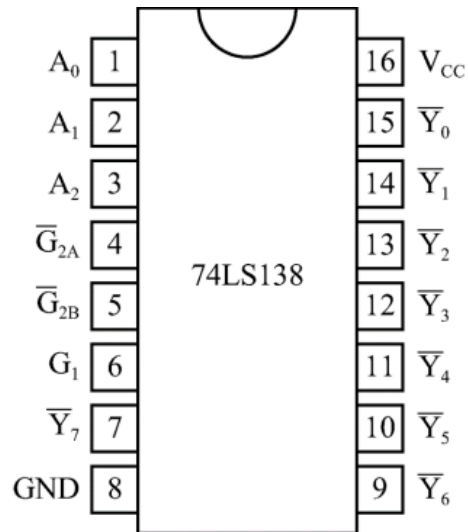
$m_0 \sim m_3$ 为 $A_1 A_0$ 的4个最小项

译码器：最小项发生器

集成二进制译码器74LS138



(a)逻辑符号



(b)引脚图

A_2 、 A_1 、 A_0 为二进制译码输入端

$\overline{Y}_7 \sim \overline{Y}_0$ 为译码输出端（低电平有效）

G_1 、 $\overline{G}_{2A}$ 、 $\overline{G}_{2B}$ 为选通控制端

当 $G_1 = 1$ 、 $\overline{G}_{2A} + \overline{G}_{2B} = 0$ 时，译码器处于工作状态；

当 $G_1 = 0$ 、 $\overline{G}_{2A} + \overline{G}_{2B} = 1$ 时，译码器处于禁止状态。

输入：自然二进制码

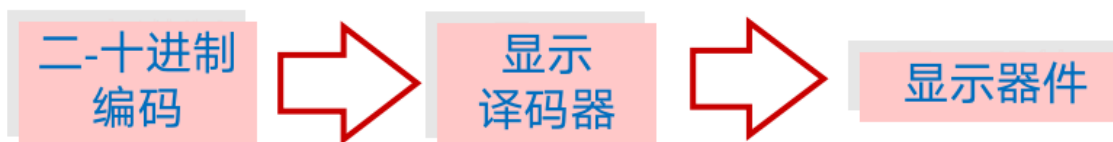
74LS138的真值表

输出：低电平有效

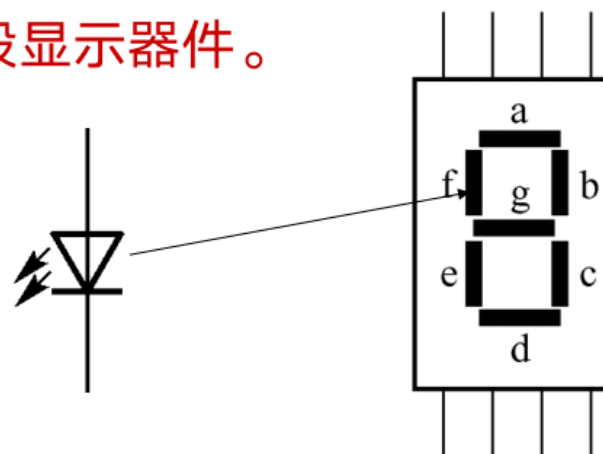
输 入						输 出							
G_1	$\overline{G}_{2A}$	$\overline{G}_{2B}$	A_2	A_1	A_0	$\overline{Y}_0$	$\overline{Y}_1$	$\overline{Y}_2$	$\overline{Y}_3$	$\overline{Y}_4$	$\overline{Y}_5$	$\overline{Y}_6$	$\overline{Y}_7$
×	1	×	×	×	×	1	1	1	1	1	1	1	1
×	×	1	×	×	×	1	1	1	1	1	1	1	1
0	×	×	×	×	×	1	1	1	1	1	1	1	1
1	0	0	0	0	0	0	1	1	1	1	1	1	1
1	0	0	0	0	1	1	0	1	1	1	1	1	1
1	0	0	0	1	0	1	1	0	1	1	1	1	1
1	0	0	0	1	1	1	1	1	0	1	1	1	1
1	0	0	1	0	0	1	1	1	1	0	1	1	1
1	0	0	1	0	1	1	1	1	1	1	0	1	1
1	0	0	1	1	0	1	1	1	1	1	1	0	1
1	0	0	1	1	1	1	1	1	1	1	1	1	0

2. 显示译码器

在数字系统中，常常需要将运算结果用人们习惯的十进制显示出来，这就要用到**显示译码器**。



显示器件： 常用的是**七段显示器件**。



(a) 数码管示意图

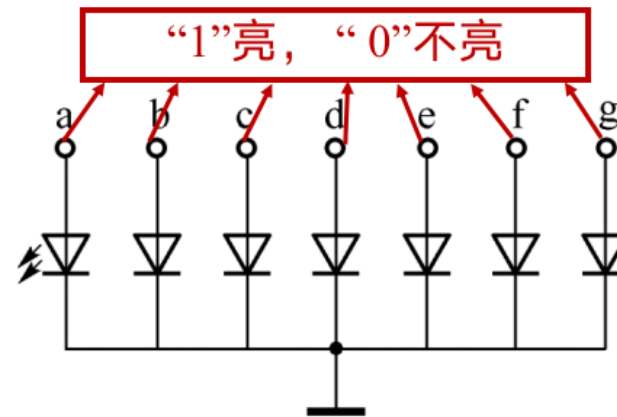
121

接法：

共阴极：

各段加**正向**电压导通，发光；

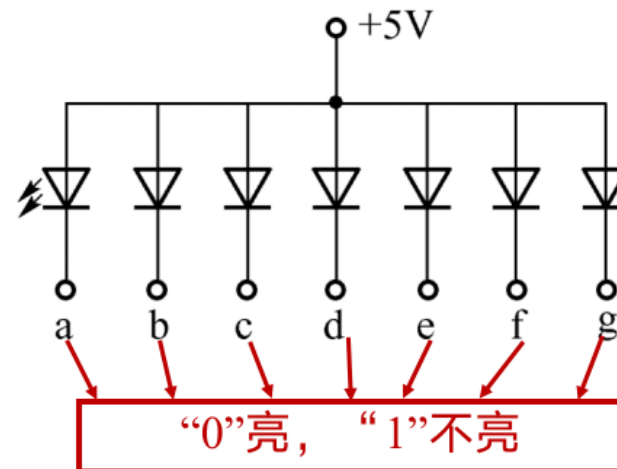
各段加**反向**电压截止，不发光。



共阳极：

各段加**反向**电压导通，发光；

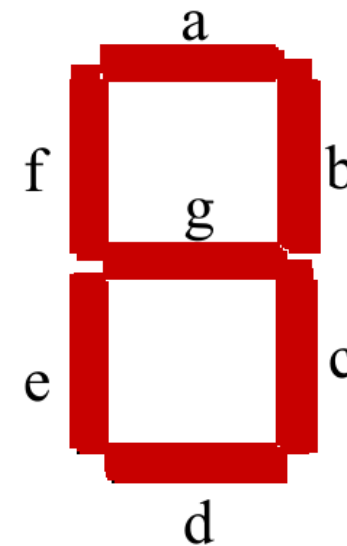
各段加**正向**电压截止，不发光。



122

七段显示译码表
(共阴极接法)

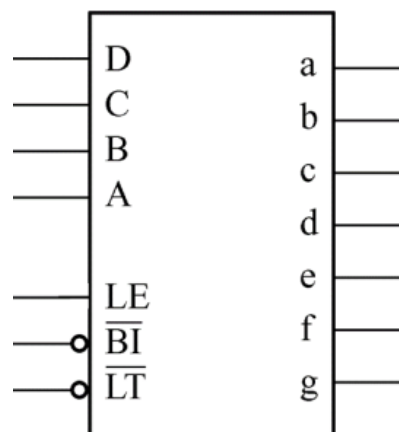
A_3	A_2	A_1	A_0	a	b	c	d	e	f	g	
0	0	0	0	1	1	1	1	1	1	0	0
0	0	0	1	0	1	1	0	0	0	0	1
0	0	1	0	1	1	0	1	1	0	1	2
0	0	1	1	1	1	1	1	0	0	1	3
0	1	0	0	0	1	1	0	0	1	1	4
0	1	0	1	0	1	1	1	0	1	1	5
0	1	1	0	1	0	1	1	1	1	1	6
0	1	1	1	1	1	1	0	0	1	0	7
1	0	0	0	1	1	1	1	1	1	1	8
1	0	0	1	1	1	1	1	0	1	1	9



123

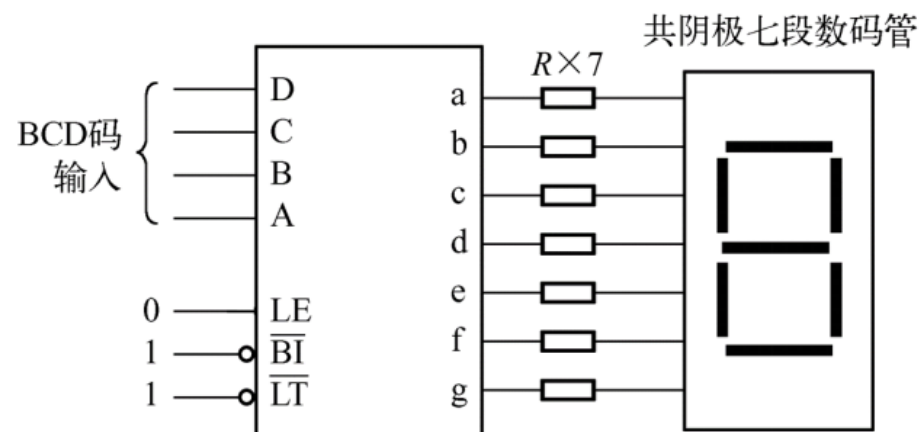
显示译码器:

CD4511的逻辑功能示意图 (驱动共阴)



- 灯测试 $\overline{\text{LT}}$: 低有效, 显示8
- 消隐输入 $\overline{\text{BI}}$: 低有效, 不显示
- 锁存 LE: 高有效, LE=1锁存, 显示数字不变。

CT4511与七段显示器件的连接



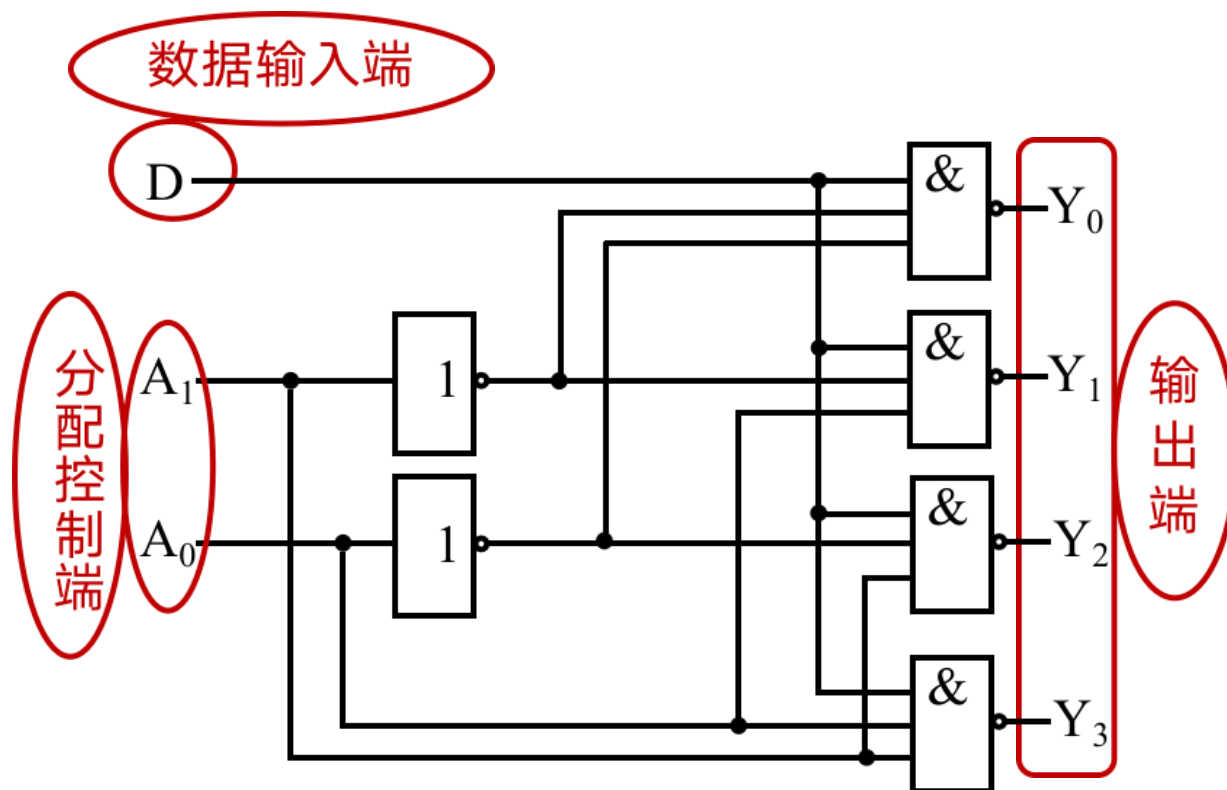
如数字万用表上的HOLD键。
按下, 可将显示数据锁存,
不再变化。



124

9.5.4 数据分配器与数据选择器

1. 数据分配器 使一路输入信号分配到多路输出。



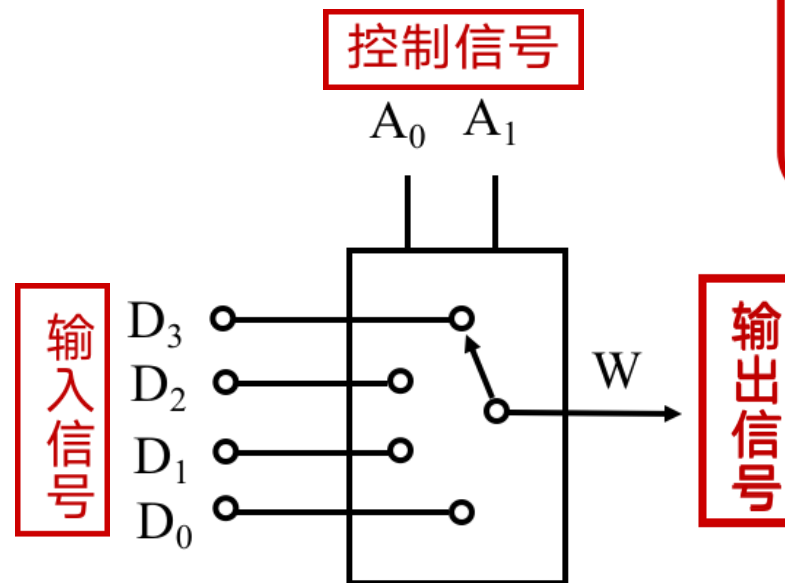
功能表

A_1	A_0	数据分配
0	0	$D \rightarrow Y_{00}$
0	1	$D \rightarrow Y_{11}$
1	0	$D \rightarrow Y_{22}$
1	1	$D \rightarrow Y_{33}$

125

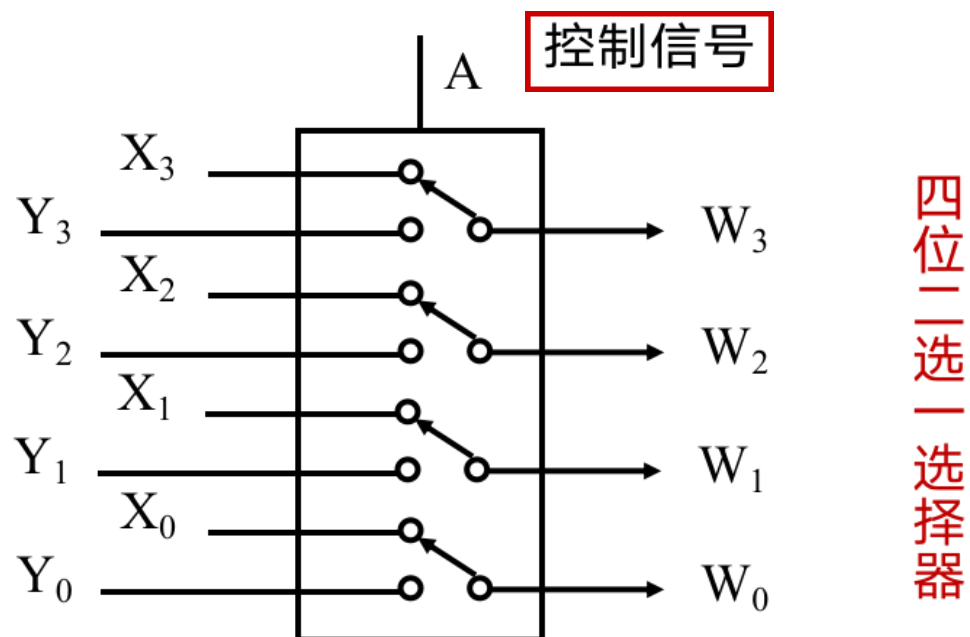
2. 数据选择器

从一组数据中选择一路信号进行传输。



数据选择器类似一个多投开关。选择哪一路信号由相应的一组控制信号控制。

从 n 个数据中选择一路传输，称为 n 选一数据选择器。若被选择数据为 m 位，则称为 m 位 n 选一数据选择器。



127

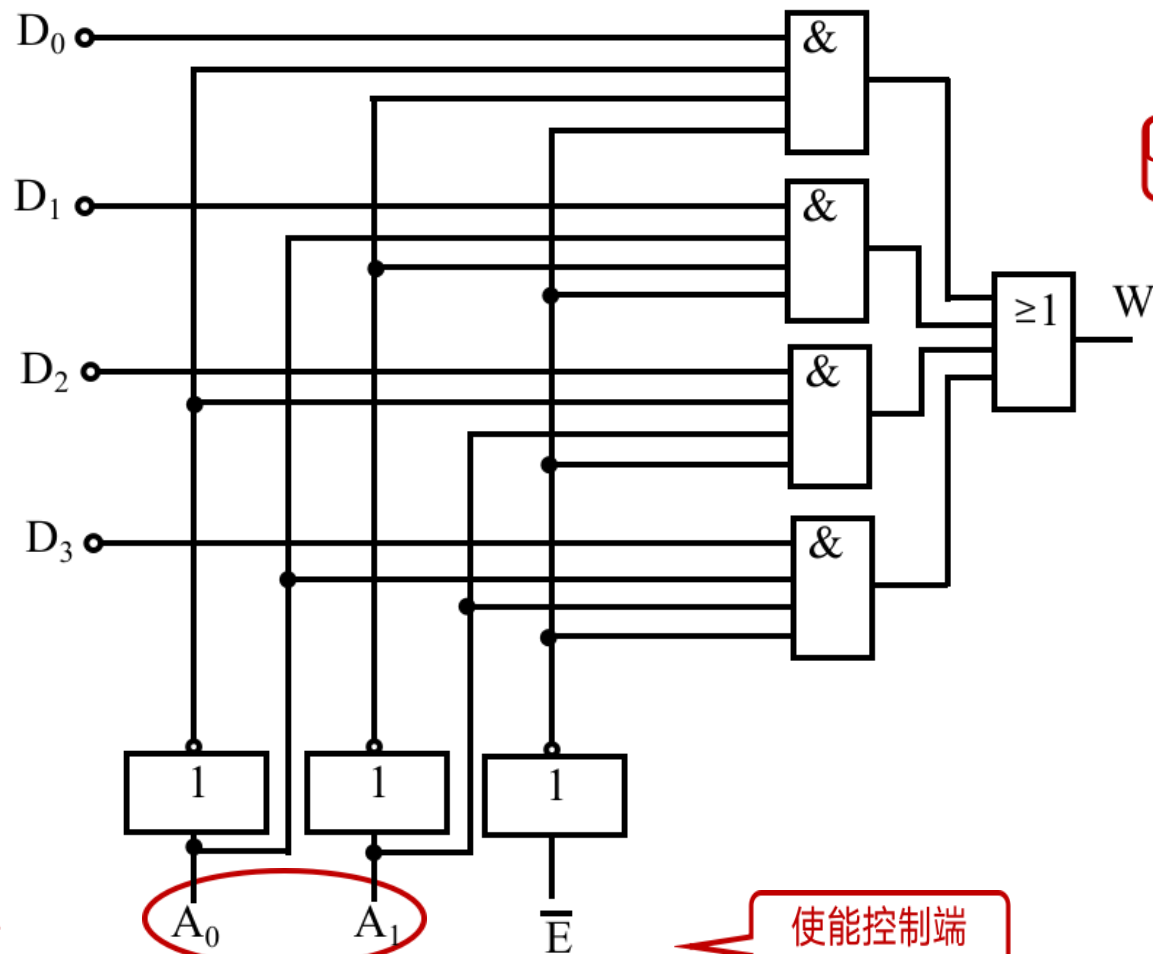
电路图

数据输入端

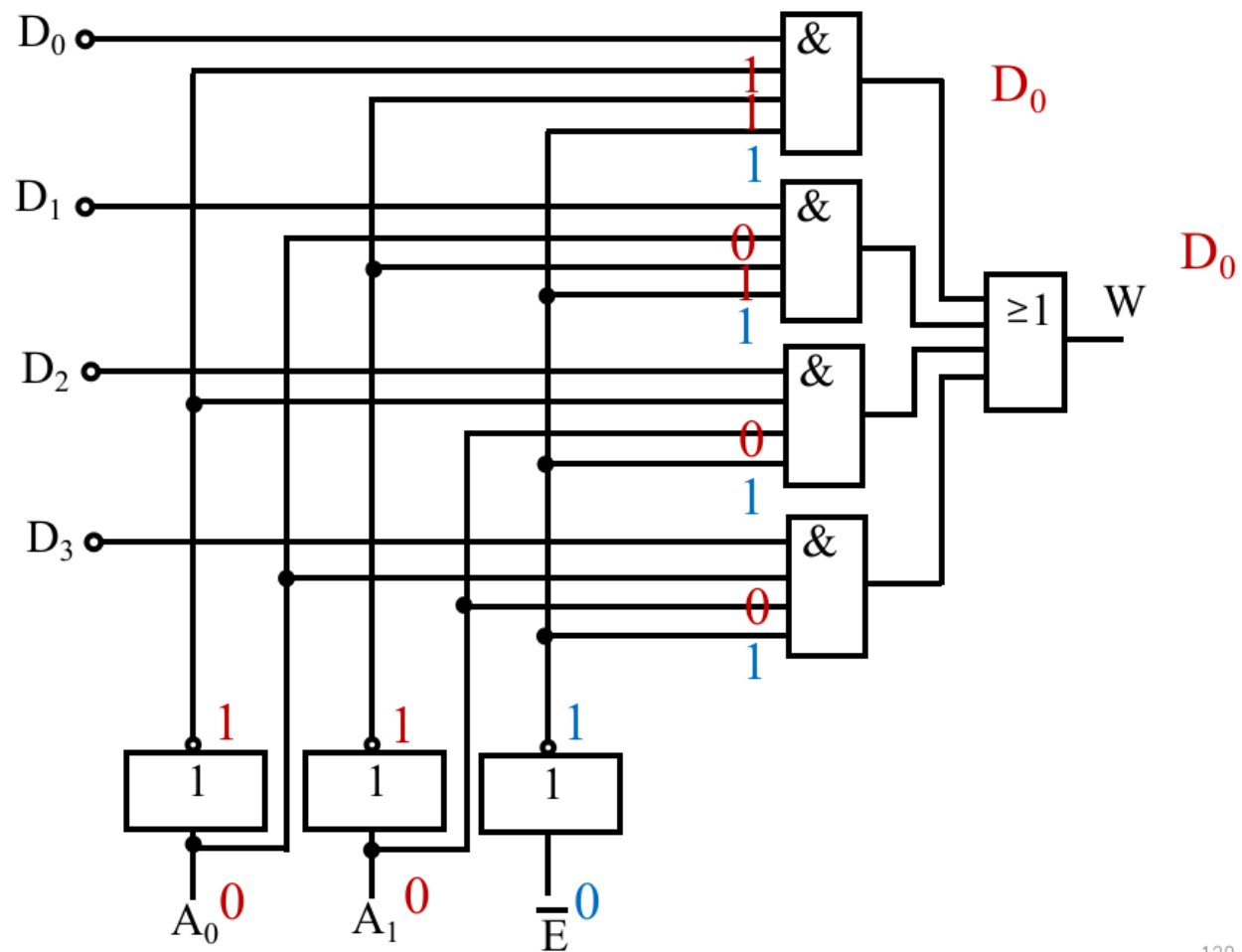
选择控制端

使能控制端

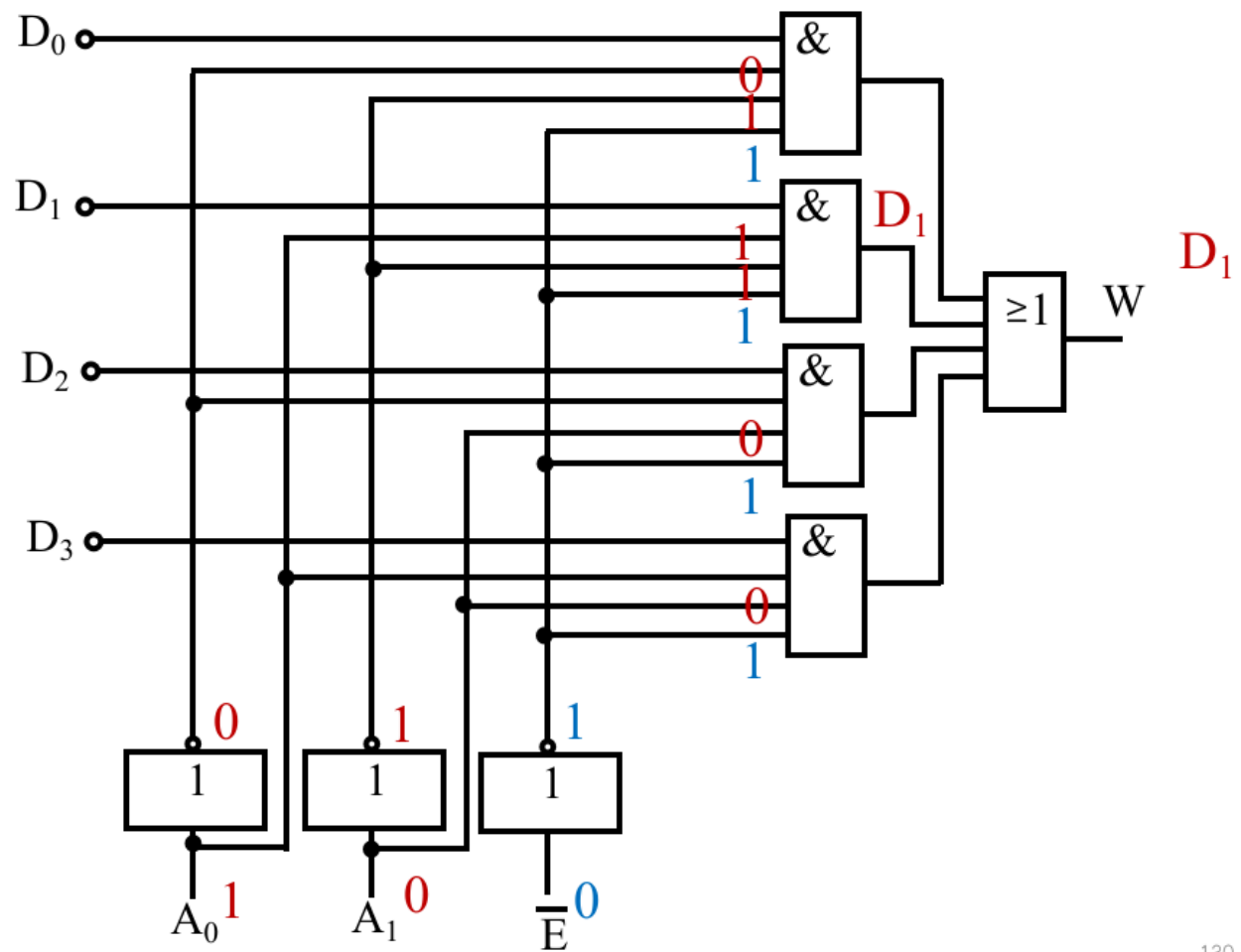
输出端



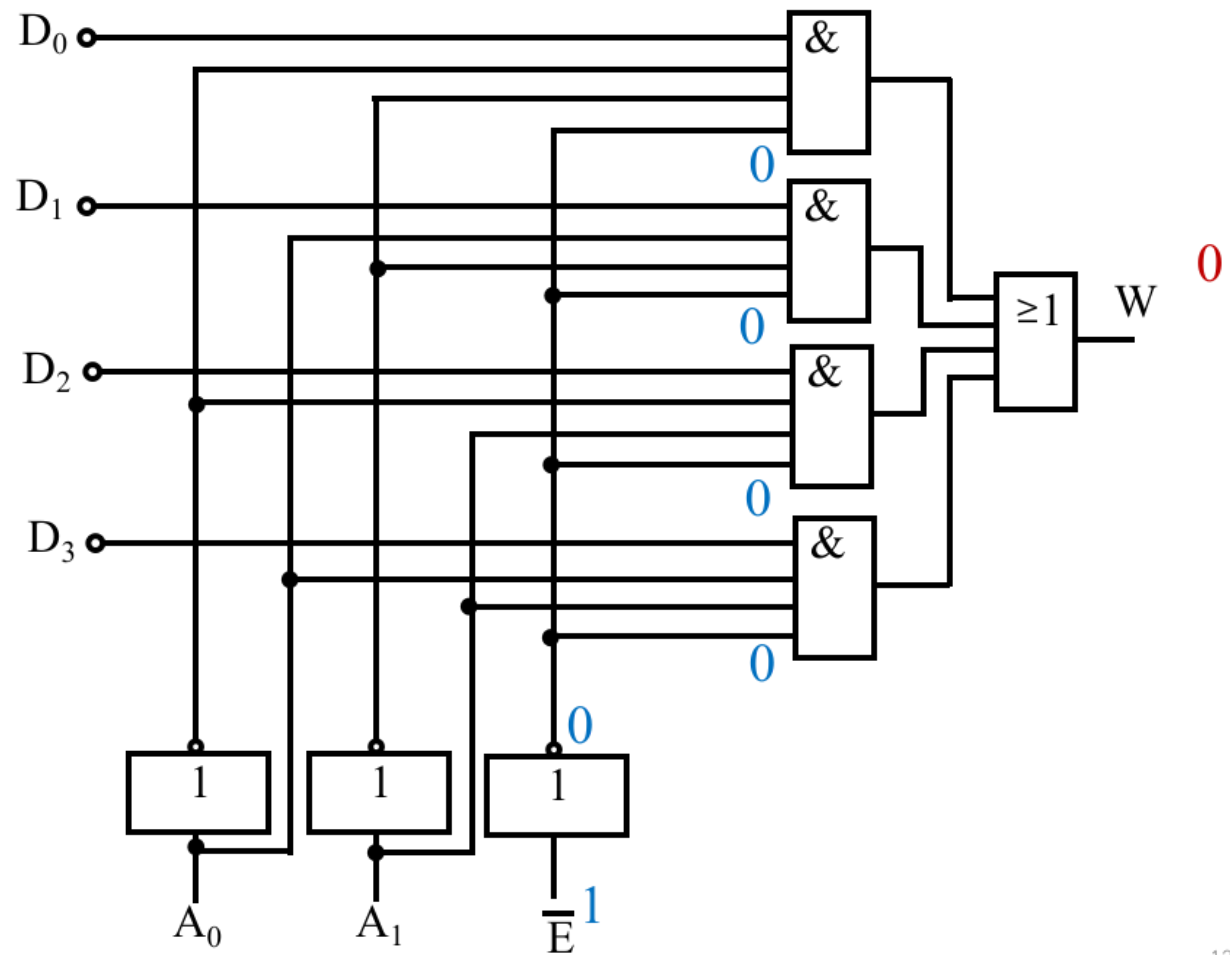
128



129

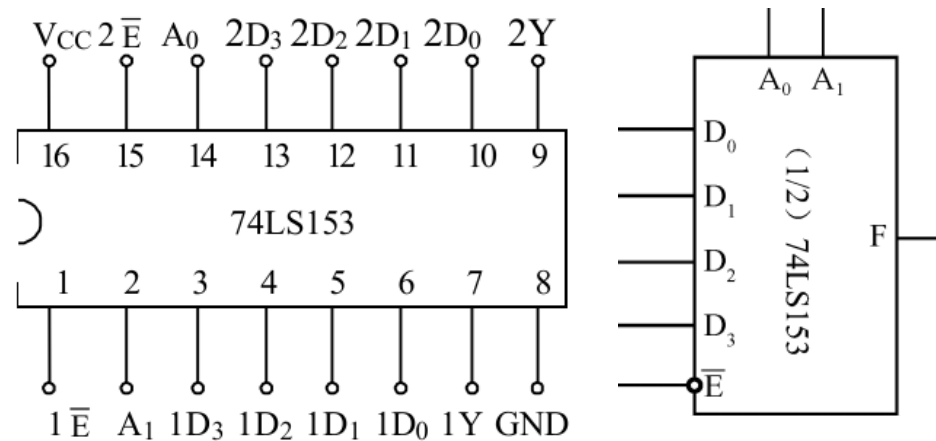


130



131

集成双4选1数据选择器74LS153



使能端	选择控制端		输 出
$\bar{E}$	A_1	A_0	F
1	$\times$	$\times$	0
0	0	0	D_0
0	0	1	D_1
0	1	0	D_2
0	1	1	D_3

$$F = \bar{A}_1 \bar{A}_0 D_0 + \bar{A}_1 A_0 D_1 + A_1 \bar{A}_0 D_2 + A_1 A_0 D_3$$



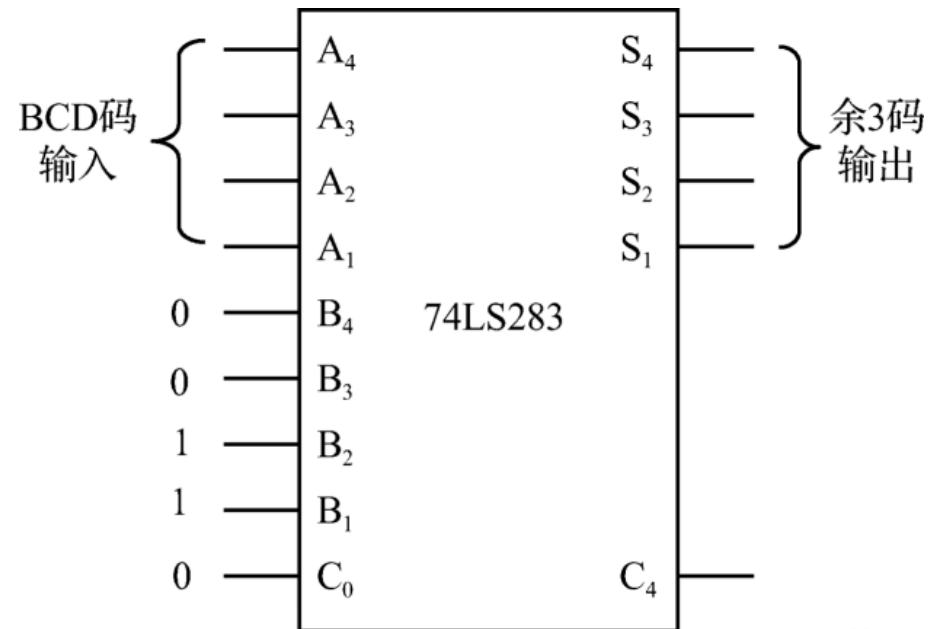
132

9.5.6 用中规模组合逻辑模块实现组合逻辑函数

1. 用全加器实现组合逻辑设计

例16 用74LS283设计一个将8421BCD码转换成余3码的转换电路。

解： BCD码+0011=余3码



133

2. 用译码器实现组合逻辑设计

实现组合逻辑函数的步骤：

- (1) 将逻辑函数的逻辑式写成最小项或的形式；
- (2) 将译码器的地址输入端接输入逻辑变量；
- (3) 将逻辑式中出现的最小项所对应的译码器输出端适当连接。
(若译码器输出端高电平有效，就将选中的输出端用或门连接；
若译码器输出端低电平有效，就将选中的输出端用与非门连接)

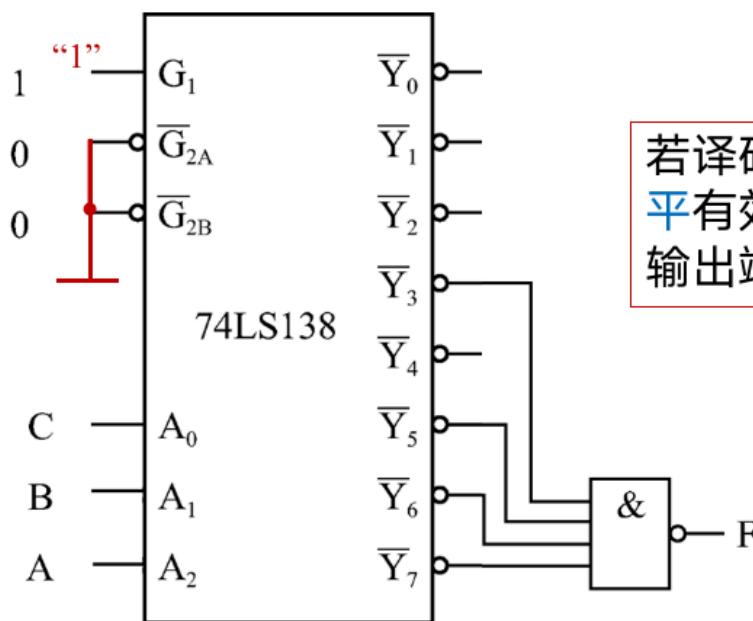
例17 利用74LS138型3-8线译码器实现三人无弃权表决电路。

解：

$$F = \overline{A}BC + A\overline{B}C + AB\overline{C} + ABC = \sum m(3,5,6,7)$$

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

三人无弃权
表决电路真值表



若译码器输出端低电平有效，就将选中的输出端用与非门连接

3. 用数据选择器实现组合逻辑设计

逻辑对比法：

只需要使数据选择器的逻辑式与要实现的逻辑函数的逻辑式相同，将输入变量直接或经门电路分别与选择输入端（地址端）及数据输入端相连，即可实现逻辑函数。

例18 利用74LS153型双四选一数据选择器实现三人无弃权表决电路。

解： 74LS153的逻辑式为：

$$F = \bar{A}_1\bar{A}_0D_0 + \bar{A}_1A_0D_1 + A_1\bar{A}_0D_2 + A_1A_0D_3$$

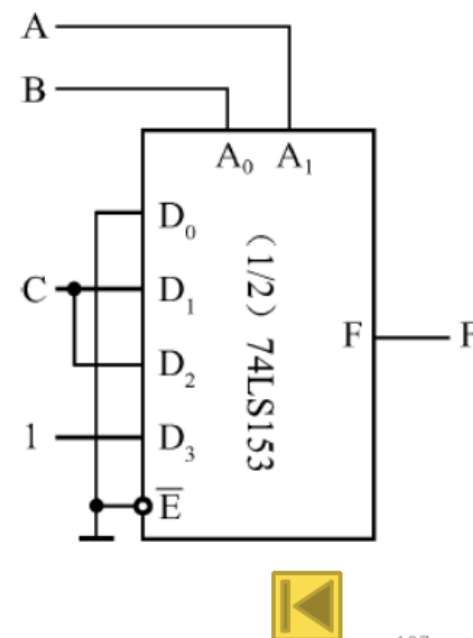
三人无弃权表决电路的逻辑函数：

$$F = \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC$$

可写成： $F = \bar{A}BC + A\bar{B}C + AB \cdot 1$

对比两个逻辑式：令 $A_1=A$ ， $A_0=B$ ，

则 $D_0=0$ ， $D_1=C$ ， $D_2=C$ ， $D_3=1$ 。



137

9.6 应用举例



138

交通信号灯故障检测电路

交通信号灯有红灯（R）、黄灯（Y）和绿灯（G）三种。正常工作时只能有一种灯亮，如果灯全亮或两个灯同时亮，则为故障状态。采用组合逻辑电路设计交通信号灯故障检测的电路。

(1) 逻辑抽象

输入变量：用R、Y、G分别表示红灯、黄灯和绿灯

0表示不亮，1表示亮。

输出变量：用F表示故障信号

0表示无故障，1表示有故障。

R	Y	G	F
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

139

(2) 写逻辑表达式

$$F = \overline{R}\overline{Y}\overline{G} + \overline{R}YG + R\overline{Y}G + RY\overline{G} + RYG$$

(3) 化简或变换

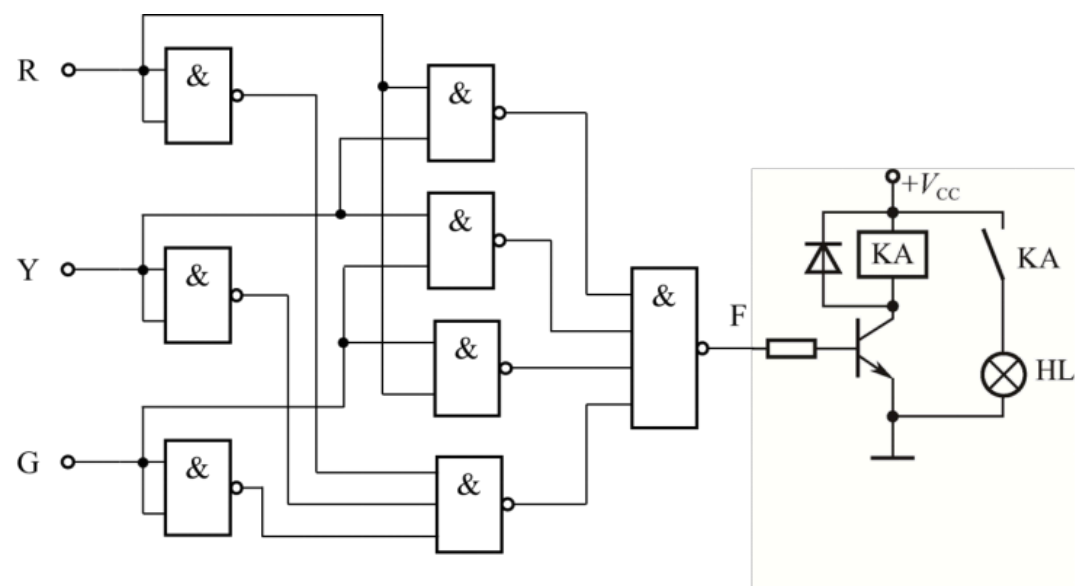
		YG			
R		00	01	11	10
	0	1		1	
	1		1	1	1

$$F = \overline{R}\overline{Y}\overline{G} + RG + YG + RY$$

140

若采用与非门实现

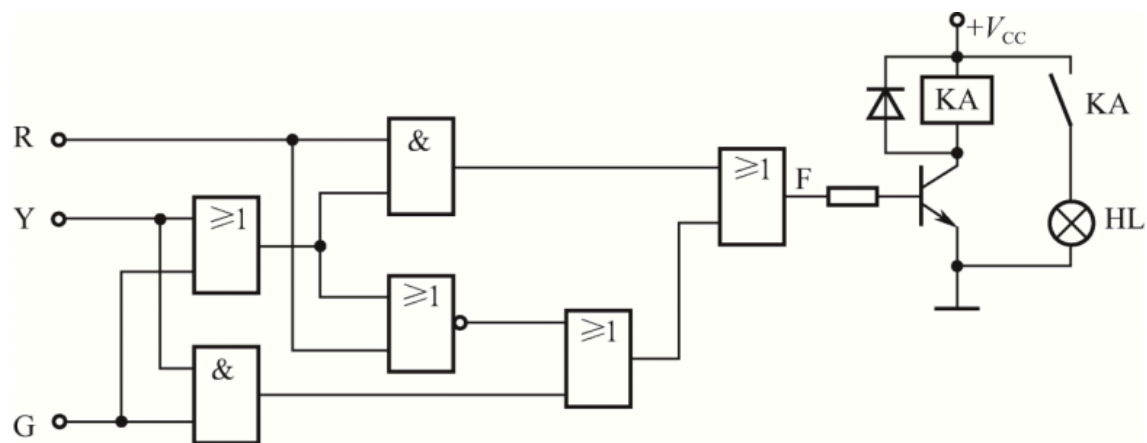
$$F = \overline{\overline{R} \overline{Y} \overline{G}} + \overline{\overline{R} \overline{G}} + \overline{\overline{Y} \overline{G}} + \overline{\overline{R} \overline{Y}} = \overline{\overline{R} \overline{Y} \overline{G}} \cdot \overline{\overline{R} \overline{G}} \cdot \overline{\overline{Y} \overline{G}} \cdot \overline{\overline{R} \overline{Y}}$$



为了减少所用门数，可将逻辑式变换

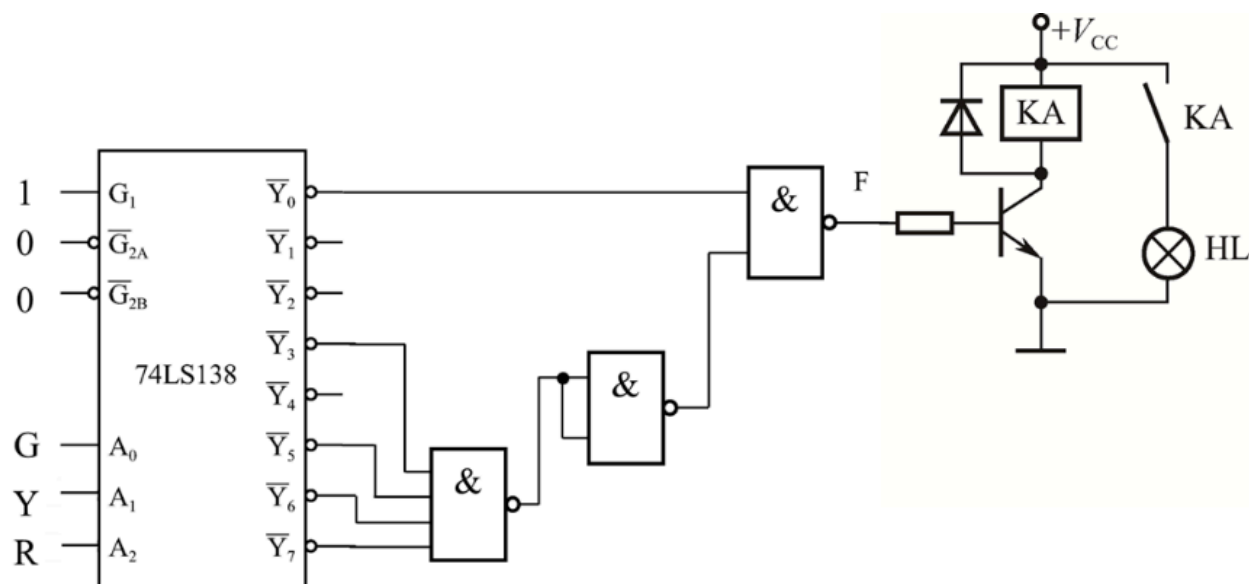
$$F = \overline{R} \overline{Y} \overline{G} + RG + YG + RY$$

$$F = \overline{\overline{R}YG} + R(G + Y) + YG = \overline{R + Y + G} + R(G + Y) + YG$$



利用3-8线译码器74LS138器件设计

$$F = \overline{R}\overline{Y}\overline{G} + \overline{R}Y\overline{G} + R\overline{Y}\overline{G} + RY\overline{G} + RY\overline{G} = \sum m(0,3,5,6,7)$$



143

利用双4选1数据选择器74LS153器件设计

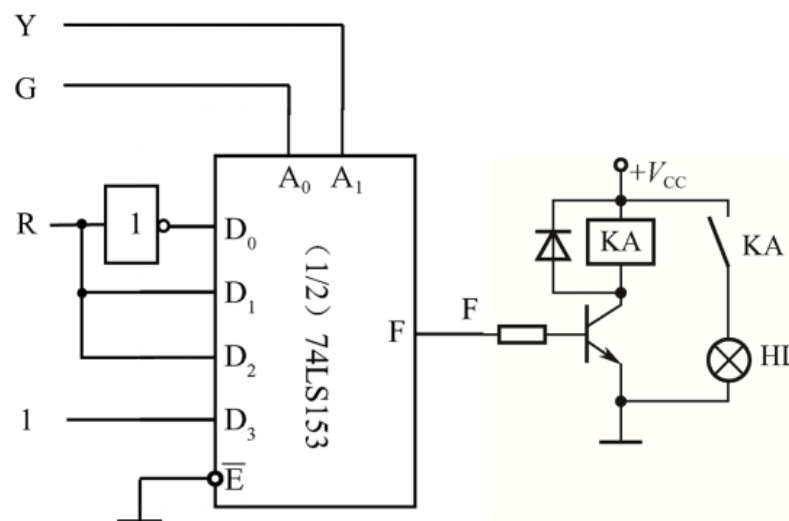
4选1数据选择器的逻辑表达式为 $F = \bar{A}_1\bar{A}_0D_0 + \bar{A}_1A_0D_1 + A_1\bar{A}_0D_2 + A_1A_0D_3$

$$F = \bar{R}\bar{Y}\bar{G} + R\bar{G} + Y\bar{G} + RY$$

对比两个逻辑式可以看出,

令 $A_1=Y$, $A_0=G$,

则 $D_0 = \bar{R}$, $D_1=D_2=R$, $D_3=1$ 。



END

145